AI-BASED MEDICAL REPORT ANALYSIS AND HEALTH INFORMATION SYSTEM

A PROJECT REPORT

Submitted by

NASMIN M (611223114063)
SANGEETHA D (611223114083)
DHARSHANKUMAR S (611223114021)
ADITHYAN K G (611223114001)

in partial fulfillment for the award the degree of

BACHELOR OF ENGINEERING

IN

MECHANICAL ENGINEERING

KNOWLEDGE INSTITUTE OF TECHNOLOGY, SALEM

ANNA UNIVERSITY: CHENNAI 600 025

MAY 2026

BONAFIDE CERTIFICATE

Certified that this project report entitled " **AI-BASED MEDICAL REPORT ANALYSIS AND HEALTH INFORMATION SYSTEM**" is the bonafide work carried out by **Nasmin M, Sangeetha D, Dharshankumar S, and Adithyan K G** in partial fulfillment of the requirements for the award of the degree of **Bachelor of Engineering in Mechanical Engineering** under Anna University, Chennai, during the academic year **2026–2027**, under our supervision.

SIGNATURE	**SIGNATURE**
Dr. K. S. PRABHARAAN, M.E., Ph.D.	Prof. S.NAVEEN KUMAR, B.E., M.E.
HEAD OF THE DEPARTMENT	SUPERVISOR
PROFESSOR	ASSISTANT PROFESSOR
DEPARTMENT OF MECHANICAL ENGINEERING	DEPARTMENT OF MECHANICAL ENGINEERING
KNOWLEDGE INSTITUTE OF TECHNOLOGY	KNOWLEDGE INSTITUTE OF TECHNOLOGY
KIOT CAMPUS	KIOT CAMPUS
KAKAPALAYAM (P.O.)SALEM – 637504	KAKAPALAYAM (P.O.)SALEM – 637504

ABSTRACT

Healthcare information is increasingly being generated in digital and document-based formats such as laboratory reports, diagnostic reports, medical images, and health records. Although these reports contain important clinical information, many users find it difficult to understand medical terminology, laboratory parameters, reference ranges, and reported abnormalities. Conventional medical reports generally present information in a technical format and require professional interpretation for proper understanding.

The proposed HealthAI – AI-Based Medical Report Analysis and Health Information System is a software-based healthcare assistance application developed to provide users with a simple and accessible method for understanding medical reports. The system combines web technologies, Optical Character Recognition (OCR), Artificial Intelligence (AI), image analysis, user authentication, multilingual support, report history, and PDF generation within a single application.

The system allows authenticated users to upload medical reports in PDF or image format. For image-based reports, an OCR module is used to extract textual information from the uploaded document. Image preprocessing techniques such as image scaling, grayscale conversion, and contrast enhancement are applied to improve OCR readability. The extracted information is then processed by an AI-based analysis module to generate a simplified explanation of the available report information.

The application also supports medical image and X-ray observation using an AI vision model. The system is designed to provide cautious visual observations without treating AI output as a confirmed medical diagnosis. When OCR values are unclear or insufficient information is available, the system is designed to avoid unsupported assumptions and indicate that the value requires verification.

A conversational HealthAI chatbot is also integrated into the application. Users can ask general health-related questions and receive concise AI-generated explanations. The system supports both English and Tamil, improving accessibility for users who prefer either language. Additional functionalities include Google-based authentication, report history management, and downloadable analysis reports in PDF format.

The proposed system therefore provides an integrated platform for medical report understanding and general health information. The project focuses on improving accessibility, readability, usability, and responsible AI-assisted interpretation while maintaining the important limitation that the system is not a replacement for qualified medical professionals.

Keywords: Artificial Intelligence, Medical Report Analysis, Optical Character Recognition, OCR, HealthAI, Healthcare Chatbot, Medical Image Analysis, X-ray Observation, React, Firebase, Groq AI, Multilingual Healthcare.

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	iii
	LIST OF TABLES	v
	LIST OF FIGURES	vi
	LIST OF ABBREVIATIONS	vii
	LIST OF NOMENCLATURE	viii
1	INTRODUCTION	1
1.1	Introduction	1
1.2	Background of the Project	2
1.3	Problem Statement	3
1.4	Existing System	3
1.5	Proposed System	4
1.6	Advantages of the Proposed System	5
1.7	Objectives of the Project	6
1.8	Scope of the Project	6
1.9	Advantages of the Proposed System	5
1.10	Objectives of the Project	6
1.11	Scope of the Project	6
1.12	Significance of the Project	7
2	LITERATURE REVIEW	8
2.1	Literature Survey Methodology	8
2.2	AI-Based Healthcare Systems	8
2.3	OCR-Based Document Processing	9
2.4	AI-Based Medical Report Interpretation	9
2.5	Medical Image Analysis	10
2.6	Healthcare Chatbots and Conversational Agents	10
2.7	Large Language Models in Health Communication	10
2.8	Multilingual Healthcare Applications	11
2.9	Existing System Comparison	11
2.10	Research Gap	12
2.11	Proposed Research Contribution	13
2.12	Summary of Literature Review	13
3	TECHNOLOGIES AND SYSTEM REQUIREMENTS	14

CHAPTER NO.	TITLE	PAGE NO.
3.1	Introduction	14
3.2	System Requirements	14
3.3	React.js	14
3.4	JavaScript	15
3.5	Tailwind CSS	15
3.6	React Router	15
3.7	Firebase Authentication	16
3.8	Node.js	16
3.9	Express.js	17
3.10	Tesseract.js	18
3.11	Groq API	19
3.12	PDF.js	20
3.13	React Markdown	20
3.14	Development Environment	21
4	SYSTEM DESIGN	22
4.1	Introduction	22
4.2	Overall System Architecture	23
4.2.1	Presentation Layer	23
4.2.2	Application Layer	23
4.2.3	AI Processing Layer	23
4.3	Overall System Workflow	24
4.4	User Interaction Flow	24
4.5	Report Processing Design	25
4.6	OCR Processing Design	26
4.7	AI Report Analysis	27
4.8	Medical Image Processing	28
4.9	HealthAI Chatbot	29
4.10	Report Information Processing	30
4.11	Result Presentation	31
4.12	PDF Generation Design	32
4.13	Multilingual Processing	33
4.14	Report History Module	34
4.15	PDF Report Generation	35
4.16	Frontend–Backend Communication	35

CHAPTER NO.	TITLE	PAGE NO.
4.17	Error Handling	36
4.18	Security Considerations	36
4.19	Working Principle Summary	36
4.20	Chapter Summary	36
5	IMPLEMENTATION	37
5.1	Introduction	37
5.2	Project Structure	37
5.3	Frontend Implementation	38
5.4	Backend Implementation	39
5.5	Authentication Implementation	39
5.6	Report Upload Implementation	40
5.7	File Processing Implementation	40
5.8	OCR Implementation	40
5.9	AI Report Analysis Implementation	41
5.10	Medical Image Implementation	41
5.11	Chatbot Implementation	41
5.12	Response Formatting	42
5.13	Multilingual Implementation	42
5.14	Report History Implementation	42
5.15	PDF Report Implementation	43
5.16	Error Handling Implementation	41
5.17	Loading State Implementation	41
5.18	Frontend–Backend Integration	42
5.19	Environment Configuration	42
5.20	Implementation Summary	43
6	RESULTS AND SYSTEM OUTPUTS	44
6.1	Introduction	44
6.2	HealthAI Home Page	45
6.3	HealthAI Login Page	46
6.4	Successful Google Authentication	47
6.5	Medical Report Upload Interface	48
6.6	OCR Processing Output	49
6.7	AI Medical Report Analysis	50
6.8	Medical Image Observation	51

CHAPTER NO.	TITLE	PAGE NO.
6.9	HealthAI Chat Interface	52
6.10	Sample HealthAI Chat Response	53
6.11	Tamil HealthAI Response	54
6.12	English HealthAI Response	54
6.13	Report History	55
6.14	PDF Output	55
6.15	Functional Results	53
6.16	Output Quality Observation	54
6.17	User Interface Observation	54
6.18	Overall System Result	55
6.19	Chapter Summary	55
7	TESTING AND PERFORMANCE ANALYSIS	56
7.1	Introduction	56
7.2	Testing Methodology	56
7.3	Functional Testing	57
7.4	Authentication Testing	58
7.5	Report Upload Testing	59
7.6	OCR Testing	60
7.7	AI Analysis Testing	60
7.8	Medical Image Testing	61
7.9	Chatbot Testing	61
7.10	Language Testing	62
7.11	History Testing	62
7.12	PDF Testing	63
7.13	Error Handling Testing	63
7.14	Integration Testing	64
7.15	Performance Observation	64
7.16	Accuracy Considerations	64
7.17	Reliability Considerations	64
7.18	Testing Limitations	65
7.19	Overall Testing Result	65
7.20	Chapter Summary	65
8	FUTURE ENHANCEMENTS	66
8.1	Introduction	66

CHAPTER NO.	TITLE	PAGE NO.
8.2	Improved OCR Accuracy	66
8.3	Advanced Medical Report Understanding	67
8.4	Advanced Medical Image Analysis	67
8.5	Longitudinal Report Comparison	68
8.6	Personalized Health Information	68
8.7	Voice-Based Interaction	68
8.8	Additional Language Support	69
8.9	Mobile Application Development	69
8.10	Improved Accessibility	69
8.11	Automatic Report Classification	70
8.12	Structured Medical Data Extraction	70
8.13	Improved PDF Generation	70
8.14	Advanced Security and Privacy	70
8.15	Cloud Deployment and Scalability	71
8.16	Healthcare Professional Integration	71
8.17	Clinical Validation	71
8.18	Future Development Roadmap	71
8.19	Chapter Summary	72
9	CONCLUSION	73
9.1	Introduction	73
9.2	Project Outcome	73
9.3	Major Achievements	73
9.4	Significance of the Developed System	73
9.5	Limitations	74
9.6	Future Scope	74
9.7	Final Conclusion	74
10	REFERENCES	75
10.1	References	75
10.2	Project Development References	76
10.3	Reference Usage	76
10.4	Chapter Summary	76

LIST OF TABLES

Table No.	Title
Table 2.1	Comparison of Existing and Proposed Approaches
Table 3.1	Software Requirements
Table 3.2	Language Processing
Table 4.1	HealthAI Module Interaction
Table 6.1	Functional Test Summary
Table 7.1	Functional Test Cases
Table 7.2	Authentication Test Cases
Table 7.3	Report Upload Test Cases
Table 7.4	OCR Test Cases
Table 7.5	AI Analysis Test Cases
Table 7.6	Medical Image Test Cases
Table 7.7	Chatbot Test Cases
Table 7.8	Language Test Cases
Table 7.9	History Test Cases
Table 7.10	PDF Test Cases
Table 7.11	Error Handling Test Cases
Table 7.12	Integration Test Summary

LIST OF FIGURES

Figure No.	Title
Figure 3.1	Firebase Authentication Workflow
Figure 3.2	OCR Processing Pipeline
Figure 3.3	Medical Image Processing Workflow
Figure 4.1	Overall HealthAI System Architecture
Figure 4.2	Overall HealthAI Workflow
Figure 4.2	Report Input Processing Flow
Figure 4.3	Medical Image / X-ray Data Flow
Figure 4.4	Complete HealthAI Workflow
Figure 6.1	HealthAI Home Page
Figure 6.2	HealthAI Login Page
Figure 6.3	Successful Google Authentication
Figure 6.4	Medical Report Upload Interface
Figure 6.5	OCR Processing Output
Figure 6.6	AI Medical Report Analysis
Figure 6.9	HealthAI Chat Interface
Figure 6.10	Sample HealthAI Chat Response
Figure 6.11	Tamil HealthAI Response
Figure 6.12	English HealthAI Response

LIST OF ABBREVIATIONS

Abbreviation	Expansion
AI	Artificial Intelligence
API	Application Programming Interface
CBC	Complete Blood Count
CORS	Cross-Origin Resource Sharing
CSS	Cascading Style Sheets
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
LLM	Large Language Model
OCR	Optical Character Recognition
PDF	Portable Document Format
RGB	Red, Green, Blue
REST	Representational State Transfer
SDK	Software Development Kit
UI	User Interface
URL	Uniform Resource Locator
X-ray	X-radiation / X-ray Imaging

LIST OF NOMENCLATURE

Term / Notation	Description
HealthAI	AI-Based Medical Report Analysis and Health Information System
Frontend	User-facing part of the HealthAI application
Backend	Server-side processing component of the application
AI Processing	Processing of report information using the configured AI service
OCR	Conversion of text from images into machine-readable text
Report Text	Textual information extracted from a medical report
Report Image	Image representation of a medical report
X-ray Image	Uploaded X-ray image used for AI-assisted observation
Chat	Conversational health-information interface
History	Storage and retrieval of previously analysed reports
PDF Output	Downloadable document containing the generated analysis
Report Type	Category or type selected for the uploaded medical report
Language	Selected response language, English or Tamil
Analysis Result	AI-generated explanation of the available report information
API Endpoint	Backend interface used for communication between frontend and backend
/api/analyze	Endpoint used for medical report analysis
/api/chat	Endpoint used for HealthAI chatbot requests
GET /	Endpoint used for server status checking
GROQ_API_KEY	Environment variable used to store the Groq API credential
openai/gpt-oss-120b	AI model configured in the current backend implementation
Pass	Status indicating that the tested function produced the expected result

SS

CHAPTER 1

INTRODUCTION

1.1 Project Background

The healthcare sector generates a large amount of information through laboratory investigations, diagnostic examinations, medical reports, prescriptions, imaging studies, and patient records. These documents contain valuable information that can assist healthcare professionals in understanding a patient's health condition. However, medical reports are generally prepared using technical terminology, abbreviations, numerical measurements, reference ranges, and specialized descriptions that may not be easily understood by ordinary users.

The rapid development of Artificial Intelligence (AI), web technologies, Optical Character Recognition (OCR), and intelligent conversational systems provides an opportunity to improve the accessibility of healthcare information. Instead of presenting medical information only in its original technical form, an intelligent software system can process the available information and provide a simplified explanation while clearly distinguishing between reported information and AI-generated interpretation.

The HealthAI – AI-Based Medical Report Analysis and Health Information System is proposed as a software application that combines these technologies into a single user-oriented platform. The system is designed to accept medical reports in PDF and image formats, extract relevant textual information, process the extracted information using AI, and present the result in a simple and readable format.

The application also provides a conversational interface through which users can ask general health-related questions. In addition, the system supports medical image and X-ray observation, multilingual interaction, report history, and downloadable analysis reports.

The current implementation uses a React-based front end, a Node.js and Express backend, Firebase authentication, Tesseract.js for OCR processing, and Groq-based AI services. The backend accepts report text, optional report images, optional X-ray images, and language selection for processing.

1.2 Background of Digital Healthcare

Digital healthcare refers to the use of software, communication technologies, data processing systems, artificial intelligence, and connected digital platforms to support healthcare-related activities.

Traditional healthcare information systems often concentrate on storing or displaying information. Modern intelligent healthcare applications are increasingly designed to assist users in understanding information and interacting with digital healthcare services.

The integration of AI into healthcare applications has expanded the possibilities for automated document processing, conversational assistance, image interpretation, information retrieval, and personalized user interfaces.

However, healthcare AI systems require careful design because incorrect interpretation of medical information may result in misunderstanding or inappropriate decisions. Therefore, the HealthAI system emphasizes clear communication, uncertainty handling, and professional medical review where appropriate.

The current HealthAI implementation explicitly instructs its image-analysis component not to diagnose diseases, claim certainty, invent findings, prescribe treatment, or provide medicine dosage. It is instead designed to provide cautious visual observations and recommend professional review when necessary.

1.3 Medical Reports and Information Accessibility

Medical reports frequently contain parameters such as hemoglobin, white blood cell count, platelet count, glucose concentration, creatinine, liver enzymes, electrolyte levels, and other laboratory measurements.

Understanding these values requires knowledge of:

- The name of the test
- The measured value
- The unit
- The laboratory reference range
- The significance of values outside the reference range
- The clinical context of the patient

A user may therefore possess a medical report without fully understanding what the information means.

The HealthAI system addresses this usability problem by processing the report and presenting important information in a simplified form. OCR is used to convert image-based reports into machine-readable text, while AI processing converts the extracted information into a user-oriented explanation.

The system also applies image preprocessing before OCR. The current OCR implementation increases image resolution, converts the image to grayscale, enhances contrast, and then performs Tesseract-based text extraction. This processing is intended to improve the quality of text recognition from uploaded medical report images.

1.4 Role of Artificial Intelligence in Healthcare

Artificial Intelligence provides computational methods for processing large amounts of information, identifying patterns, generating natural-language responses, and assisting users with complex information.

In the proposed HealthAI system, AI is used for multiple purposes:

1. Medical report text analysis
2. Medical image and X-ray observation
3. General health question answering
4. Multilingual response generation

5. Simplification of technical information

The AI system is not intended to replace healthcare professionals. Instead, it functions as an information-assistance layer between the original medical information and the user.

The backend supports both English and Tamil response modes. The current implementation instructs the AI to keep laboratory values, numbers, and units exactly as provided and to produce concise responses.

1.5 Problem Statement

Medical reports are often difficult for non-specialist users to understand because they contain technical terminology, numerical values, abbreviations, and reference ranges. Users may search individual parameters on different websites or depend on external sources to understand their reports.

Existing general-purpose systems may also provide lengthy explanations, make assumptions when information is incomplete, or fail to distinguish clearly between extracted information and interpretation.

Another challenge is that many medical reports are available as scanned documents or images. Before such information can be analysed computationally, the text must first be extracted using OCR technology. Poor image quality, unusual layouts, small fonts, handwritten content, and scanning artifacts can reduce OCR accuracy.

Therefore, there is a need for an integrated system that can:

- Accept medical reports in common digital formats
- Extract text from report images
- Process the extracted information using AI
- Provide simple explanations
- Handle unclear information without guessing
- Support medical image observation
- Provide general health information through a chatbot
- Support multiple languages
- Maintain a history of previously analysed reports
- Generate downloadable analysis reports

1.6 Existing System

In a conventional workflow, a user receives a medical report and manually reads the document. If the terminology is unfamiliar, the user may search individual terms through search engines, medical websites, or other information sources.

For image-based reports, the user may first need to manually read the values because the document is not directly machine-readable. This creates additional difficulty when the report contains many parameters.

General AI chat systems can explain medical terminology when the user manually enters the relevant information. However, this process requires the user to identify and copy the important information from the report.

The proposed HealthAI system attempts to combine these individual activities into a single application workflow.

1.7 Limitations of Existing System

The limitations identified for the conventional approach include:

1. Manual reading of medical reports.
2. Difficulty understanding technical terminology.
3. Repeated manual searching of individual test parameters.
4. Difficulty extracting information from image-based reports.
5. Lack of a unified report-analysis workflow.
6. Lack of integrated report history in a simple user interface.
7. Limited multilingual accessibility in many systems.
8. Requirement to manually transfer report information into general AI systems.
9. Difficulty obtaining a concise explanation from lengthy medical information.
10. Lack of an integrated chatbot and report-analysis environment.

1.8 Proposed System

The proposed HealthAI system provides an integrated web-based platform for medical report understanding and general health information.

The major workflow is:

User Authentication → Report Upload → OCR/Text Extraction → Report Type Detection → AI Analysis → Result Presentation → History Storage → PDF Download

For image-based reports, the original image can additionally be passed to the AI vision component to help reduce errors caused by OCR. The current backend accepts report Text, report Image, x-ray Image, and language as analysis inputs.

The system also includes a separate Health AI chatbot that allows users to ask general health questions.

1.9 Advantages of the Proposed System

The proposed system provides the following advantages:

- Simple web-based interface
- PDF and image report support
- OCR-based text extraction
- AI-assisted report explanation
- Medical image/X-ray observation
- General health chatbot
- English and Tamil language support
- Report history
- PDF download
- Centralized user workflow
- Reduced need for manual copying of report information
- Improved readability of technical information
- Uncertainty handling for unclear image information

The system's image-processing instructions specifically prioritize visible information and avoid unsupported conclusions.

1.10 Objectives of the Project

The primary objectives of the HealthAI project are:

1. To develop a web-based AI-assisted health information application.
2. To enable users to upload medical reports in PDF and image formats.
3. To extract text from medical report images using OCR.
4. To preprocess report images to improve OCR readability.
5. To analyse extracted medical information using an AI model.
6. To provide concise and understandable explanations.
7. To provide general health information through a chatbot.
8. To support English and Tamil language interaction.
9. To provide cautious observation of uploaded medical images and X-rays.
10. To maintain a history of analysed reports.
11. To provide downloadable analysis reports.
12. To improve accessibility and usability of medical information while maintaining appropriate limitations regarding professional medical diagnosis.

1.11 Scope of the Project

The scope of the Health AI system includes:

- User authentication
- Medical report upload
- PDF processing
- Image preprocessing
- OCR
- Report type detection
- AI report analysis
- Medical image observation
- X-ray observation
- Health chatbot
- Multilingual output
- Report history
- PDF generation
- User-friendly presentation

The system is intended primarily as an information and explanation tool. It does not replace professional medical diagnosis, treatment planning, or clinical decision-making.

1.12 Significance of the Project

The significance of HealthAI lies in its integration of multiple technologies into a single healthcare-oriented application.

Instead of requiring users to separately perform OCR, search medical terms, use an AI chatbot, save report results, and create a document, HealthAI combines these activities into one workflow.

The project also demonstrates the practical application of modern software technologies such as React, Node.js, Express, Firebase, OCR, and generative AI in a healthcare-information context.

CHAPTER 2

LITERATURE REVIEW

2.1 Literature Survey Methodology

The literature review for the HealthAI project focuses on the application of Artificial Intelligence (AI), Optical Character Recognition (OCR), conversational agents, Large Language Models (LLMs), and medical image analysis in healthcare.
The review was used to understand existing approaches, identify their advantages and limitations, and determine the research gap addressed by the proposed HealthAI system.
The major areas considered are:

1. AI-based healthcare applications
2. OCR-based medical document processing
3. AI-assisted medical information interpretation
4. Medical image and radiology analysis
5. Healthcare chatbots and conversational agents
6. Multilingual healthcare assistance
7. Safety and responsible use of AI in healthcare

Recent systematic reviews show that healthcare AI research is expanding rapidly, but evaluation quality, safety, transparency, and clinical validation remain important considerations.

2.2 AI-Based Healthcare Systems

Artificial Intelligence has become an important research area in healthcare because of its ability to process large quantities of information and support healthcare-related tasks.
AI-based healthcare systems have been investigated for applications including clinical decision support, health education, patient communication, monitoring, and information retrieval.
The use of AI in healthcare does not necessarily require replacing healthcare professionals. Instead, AI can be designed as an assistance layer that helps users access and understand information.
For the HealthAI project, this approach is particularly relevant because the system is designed as an **information-assistance platform rather than an autonomous diagnostic system**.

The proposed application therefore concentrates on:

- Information extraction
- Information explanation

- User interaction
- General health education
- Report summarization
- Image observation

This approach also reduces the risk of presenting an AI-generated response as a confirmed medical diagnosis.

2.3 OCR-Based Document Processing

Medical information is frequently available in scanned documents, printed laboratory reports, and images. Such documents cannot always be directly processed as structured digital data.

Optical Character Recognition provides a method for converting text present in an image into machine-readable characters.

In the proposed HealthAI system, OCR forms an important part of the report-processing pipeline.

The workflow consists of:

Medical Report Image → Image Preprocessing → OCR → Extracted Text → AI Processing

The current implementation uses **Tesseract.js** for OCR. Before recognition, the image can be enlarged, converted to grayscale, and processed for contrast enhancement. This preprocessing is intended to improve the readability of characters before OCR is performed.

However, OCR is not completely reliable for every document. Recognition quality may be affected by:

- Low image resolution
- Blurred text
- Uneven lighting
- Complex layouts
- Handwritten content
- Small fonts
- Scanned document quality
- Symbols and medical abbreviations

Therefore, the HealthAI system should not blindly treat every OCR output as correct. Unclear values should be identified and verified against the original report.

2.4 AI-Based Medical Report Interpretation

Laboratory reports contain numerical values, units, reference ranges, abbreviations, and test names. The interpretation of these values requires appropriate context.

Generative AI and LLM-based systems can transform technical information into natural-language explanations. This makes them potentially useful for patient-oriented education and information accessibility.

However, research also highlights important limitations. A systematic review of LLM-based health-advice studies found considerable variation in how performance was evaluated and reported, with relatively limited attention to ethical, regulatory, and patient-safety considerations in many studies.

Therefore, HealthAI follows a constrained approach:

- Use the information available in the report.
- Avoid inventing missing values.
- Distinguish unclear OCR values.
- Avoid unsupported diagnoses.
- Explain findings in simple language.
- Recommend professional consultation when appropriate.

This design allows the AI component to focus on **explanation rather than autonomous clinical decision-making**.

2.5 Medical Image Analysis

Artificial Intelligence has been extensively investigated for medical imaging applications, particularly in radiology.

A systematic review of AI in radiology identified a large body of research involving deep learning and medical imaging. However, the review also noted substantial variation in study design and methodological quality.

More recent research on responsible AI in medical imaging emphasizes that accuracy alone is not sufficient. Transparency, uncertainty, privacy, fairness, and clinical trust are also important considerations.

HealthAI incorporates medical image and X-ray processing as an **observation-support feature** rather than presenting it as a definitive diagnostic system.

The intended workflow is:

Image Upload → AI Vision Processing → Visible Observation → User Explanation

If the image quality is insufficient or the finding cannot be reliably determined, the system should communicate uncertainty rather than generate a confident conclusion.

2.6 Healthcare Chatbots

Healthcare chatbots provide a conversational interface through which users can ask health-related questions.

Earlier systematic reviews found that healthcare conversational agents have been investigated for areas including health education, monitoring, treatment support, screening, and behaviour-related interventions. Evidence regarding effectiveness and

usability has generally been positive or mixed, while limitations related to study quality, privacy, security, and evaluation have also been identified.

A systematic review specifically examining healthcare conversational agents for chronic conditions found promising user acceptance, with helpfulness, satisfaction, and ease of use reported in more than half of the included studies.

More recent research continues to identify safety, privacy, fairness, trustworthiness, usefulness, and operational effectiveness as important areas for healthcare chatbot evaluation.

The HealthAI chatbot is therefore designed primarily for **general health information and education**. It is not intended to replace a physician or provide autonomous diagnosis.

2.7 Large Language Models in Health Communication

Large Language Models have increased the ability of conversational systems to generate natural-language responses.

Recent systematic evidence suggests that LLM-based tools can potentially improve the clarity, empathy, and information richness of health-related communication. However, the same research also identifies concerns such as excessive response length, readability, inaccuracies, and the need for professional oversight.

Another recent systematic review and meta-analysis of AI chatbots in patient education reported generally high appropriateness of chatbot responses but also identified relatively high reading levels as a limitation for patient-facing use.

This finding is directly relevant to HealthAI because the project aims to provide **short, simple, and readable explanations** rather than unnecessarily long technical responses.

2.8 Multilingual Healthcare Applications

Language accessibility is an important consideration when designing healthcare information systems.

A system that provides information only in English may be less accessible to users who are more comfortable communicating in regional languages.

HealthAI therefore incorporates **English and Tamil language support**.

The system allows the user to select the preferred language before processing a report or interacting with the AI system.

The multilingual processing approach is designed to preserve:

- Laboratory values
- Numerical values
- Units
- Reference ranges
- Medical terminology

while translating the explanatory portion into the selected language.

This allows the original technical information to remain identifiable while making the explanation easier to understand.

2.9 Existing System Comparison

Table 2.1 – Comparison of Existing and Proposed Approaches

Feature	Conventional Approach	HealthAI
Medical report upload	Manual handling	Integrated upload
Image report text extraction	Manual reading	OCR-based extraction
Report explanation	Manual searching	AI-assisted explanation
Chat-based health information	Separate service required	Integrated chatbot
X-ray/image observation	Separate analysis system	Integrated observation feature
Language support	Often limited	English and Tamil
Report history	Manual storage	Application-based history
PDF analysis output	Manual preparation	Downloadable PDF
Unclear OCR values	May be manually interpreted	Marked for verification
User interface	Multiple tools may be required	Unified application

The major characteristics of conventional approaches and the proposed HealthAI approach can be summarized as follows

2.10 Research Gap

The literature indicates that AI, OCR, conversational agents, and medical image analysis have each been extensively studied as individual technologies.

However, several challenges remain when these technologies are combined into a single user-oriented healthcare information platform.

The identified gaps include:

1. Fragmented User Workflow

Users may need separate systems for OCR, report interpretation, chatbot-based questions, and report storage.

2. Readability

AI-generated healthcare information may become lengthy or difficult to understand. Recent research has specifically identified readability as an important limitation of chatbot-based patient education.

3. Uncertainty Handling
OCR and AI systems can produce incorrect results when the original document or image is unclear. Therefore, uncertainty needs to be explicitly communicated rather than hidden.

4. Responsible Medical Image Interpretation
Medical imaging systems require more than raw prediction accuracy. Responsible AI research highlights uncertainty, transparency, privacy, fairness, and clinical trust as important considerations.

5. Multilingual Accessibility
Many healthcare AI systems do not provide a unified multilingual user experience. HealthAI addresses this requirement by providing English and Tamil interaction.

6. Integrated Report Management
A user-oriented platform can provide additional value by combining report upload, analysis, history, and downloadable output within one workflow.

2.11 Proposed Research Contribution
Based on the identified research gap, the HealthAI project contributes an integrated software platform that combines:

OCR + AI Report Analysis + Medical Image Observation + Healthcare Chatbot + Multilingual Support + Report History + PDF Generation

The primary contribution is not the development of a new AI model. Instead, the project focuses on the **integration and user-oriented implementation of existing technologies** into a single healthcare information application.
The system also emphasizes responsible output generation by distinguishing between available information, unclear information, and AI-generated explanations.

2.12 Summary of Literature Review
The literature review demonstrates that AI-based healthcare applications, OCR, medical image analysis, and conversational agents have significant potential for improving healthcare information accessibility.
At the same time, existing research emphasizes the importance of accuracy, readability, safety, privacy, transparency, and professional oversight.
The HealthAI project addresses these considerations through an integrated web-based application that combines medical report processing, OCR, AI explanation, image

observation, chatbot interaction, multilingual support, report history, and PDF generation.

The literature review therefore provides the technical and conceptual foundation for the system architecture and implementation described in the following chapters.

CHAPTER 3

TECHNOLOGIES AND SYSTEM REQUIREMENTS

3.1 Introduction

The HealthAI system is developed as a web-based intelligent healthcare information application. The system combines front-end development, backend API services, user authentication, Optical Character Recognition (OCR), Artificial Intelligence (AI), medical image processing, multilingual response generation, report history, and PDF generation.

The architecture is divided into three major layers:

1. **Presentation Layer** – provides the user interface and user interaction.
2. **Application Layer** – handles authentication, report processing, API communication, and application logic.
3. **AI Processing Layer** – performs OCR-based text extraction, medical report analysis, chatbot response generation, and image observation.

The technologies selected for the project are based on their suitability for rapid web application development, API integration, OCR processing, authentication, and AI-assisted information processing.

3.2 System Requirements

The system requirements are divided into:

- Software requirements
- Hardware requirements
- Development requirements
- External service requirements

Table 3.1 – Software Requirements

Requirement	Technology
Front-End Framework	React.js
Programming Language	JavaScript
Styling	Tailwind CSS
Routing	React Router
Backend Runtime	Node.js
Backend Framework	Express.js
Authentication	Firebase Authentication
OCR	Tesseract.js
AI Service	Groq API
Data Format	JSON
Report Format	PDF / Image
Version Control	Git
Development Environment	Visual Studio Code

3.3 React.js

React.js is used as the primary front-end framework for the HealthAI application.

The user interface consists of multiple functional pages, including:

- Login
- Registration
- Home
- Reports
- Chat
- History
- Profile

React supports component-based development, allowing individual parts of the application to be developed and maintained independently.

The HealthAI application uses React state management for handling:

- Selected files
- Report previews
- Analysis results
- Loading states
- Chat messages
- Language selection
- Authentication-related UI states

This structure allows the application interface to respond dynamically to user actions.

3.4 JavaScript

JavaScript is used as the primary programming language for both the front-end and backend portions of the application.

In the front end, JavaScript handles:

- User interaction
- File selection
- State management
- API communication
- Dynamic rendering
- Navigation
- Chat interaction

In the backend, JavaScript is used with Node.js and Express.js to implement REST API endpoints and communicate with the Groq AI service.

3.5 Tailwind CSS

Tailwind CSS is used to design the user interface.

The application uses utility classes for:

- Spacing
- Typography
- Colors
- Borders

- Shadows
- Responsive layouts
- Buttons
- Cards
- Forms
- Chat bubbles

The use of utility-based styling allows the interface to be developed consistently across different pages.

The HealthAI interface follows a simple visual design with rounded cards, clear headings, accessible buttons, and separate sections for uploaded reports and AI-generated results.

3.6 React Router

React Router is used for navigation between application pages.

The major application routes include:

Route	Purpose
/	Home page
/login	Login page
/register	Registration page
/reports	Medical report analysis
/chat	HealthAI chatbot
/history	Previous report history
/profile	User profile

Protected routes are controlled using Firebase authentication status.

If a user is not authenticated and attempts to access a protected page, the application redirects the user to the login page.

3.7 Firebase Authentication

Firebase Authentication is used to provide secure user authentication.

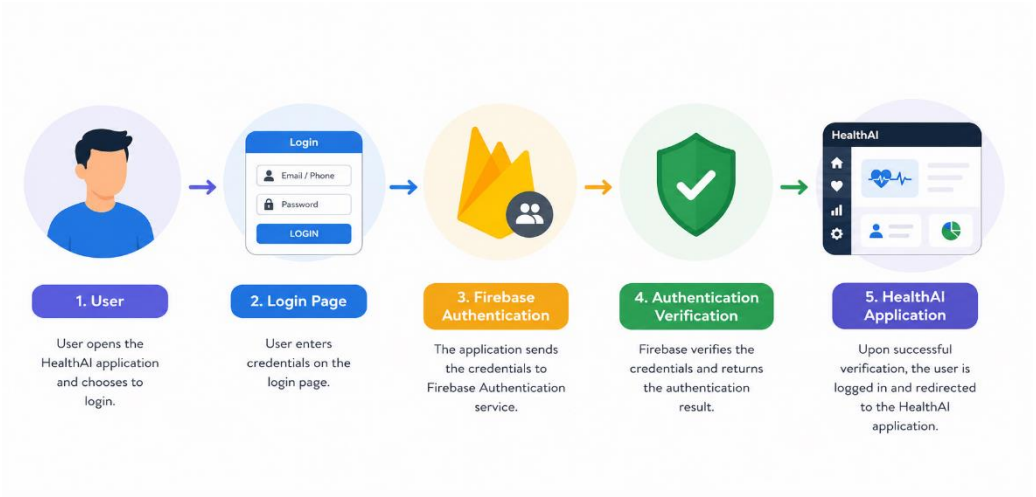
The implemented authentication system supports Google-based sign-in.

Authentication provides the following benefits:

- User identity management
- Protected application routes
- Persistent login sessions
- Controlled access to user-specific features

The application monitors the authentication state and updates the interface according to whether a valid authenticated user is available.

Figure 3.1 – Firebase Authentication Workflow



3.8 Node.js

Node.js provides the runtime environment for the HealthAI backend.
It enables JavaScript to be executed on the server side and provides the environment required for:

- API development
- Request processing
- AI service communication
- Error handling
- JSON data processing

The backend server runs locally during development on port **5000**.

3.9 Express.js

Express.js is used as the backend web framework.
The HealthAI backend exposes API endpoints for the major AI functionalities.
The primary endpoints are:

API Endpoint	Purpose
GET /	Server status check
POST /api/analyze	Medical report analysis
POST /api/chat	HealthAI chatbot

The backend receives data from the React application, validates the input, communicates with the AI service, and returns the generated response to the front end.

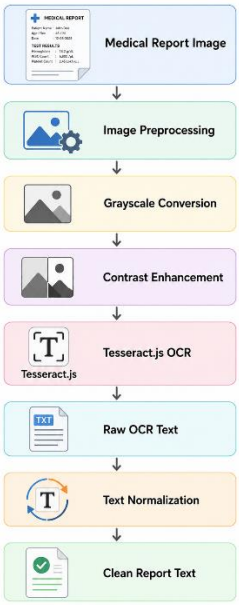
3.10 Tesseract.js

Tesseract.js is used for Optical Character Recognition.
OCR converts text contained in an image into machine-readable text.

The HealthAI OCR process includes:

1. Image selection
2. Image loading
3. Image scaling
4. Grayscale conversion
5. Contrast enhancement
6. OCR processing
7. Text extraction
8. Text normalisation

Figure 3.2 – OCR Processing Pipeline



Clean Report Text

The current OCR implementation also normalises decimal formatting and removes excessive spaces without intentionally changing the medical information extracted from the report.

3.11 Image Preprocessing

Image preprocessing is performed to improve OCR performance.

The implementation includes image scaling to increase the report's effective resolution before OCR.

The image is also converted to grayscale using weighted RGB values. Contrast enhancement is then applied to improve the distinction between text and background.

The preprocessing pipeline is:

Original Image → Scaling → Grayscale → Contrast Enhancement → OCR

This approach is particularly useful for scanned or photographed reports in which the text may not have sufficient contrast.

However, preprocessing cannot guarantee correct OCR for every image. Therefore, the system should continue to treat uncertain OCR values cautiously.

3.12 Groq AI

Groq API is used as the AI processing service for HealthAI.

The backend communicates with the Groq service using the Groq SDK.

The system uses the model configured in the current backend implementation:

Open ai/gpt-oss-120b

The model is used for:

- Medical report analysis
- General health information
- Chatbot responses
- Multilingual responses

The backend sends a system instruction together with the user's report information or chat message.

The report-analysis endpoint accepts:

- Report Text
- Report Image
- X-ray Image
- language

This allows the application to provide both OCR-derived information and optional image information to the AI processing layer.

3.13 AI-Based Report Analysis

The report-analysis module processes extracted report information using the AI service.

The AI instructions emphasise:

- No unsupported diagnosis
- No invented values
- Use of available report information
- Identification of unclear information
- General health explanation
- Professional medical consultation when appropriate

The system is therefore intended to function as an **information explanation tool** rather than an autonomous clinical decision-making system.

3.14 Medical Image and X-ray Processing

Health AI includes image analysis capabilities for uploaded medical images and X-rays.

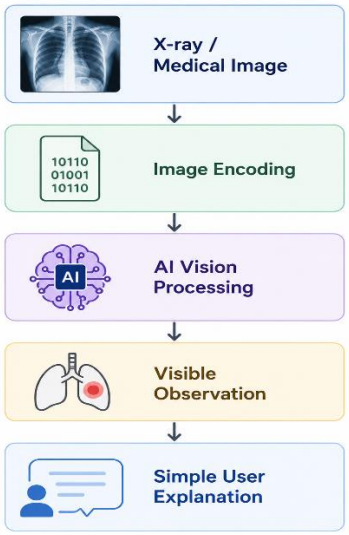
The image-analysis component is instructed to:

- Describe only visible findings
- Avoid unsupported conclusions
- Avoid disease diagnosis

- Avoid treatment recommendations
- State when the image is unclear
- Recommend professional review where appropriate

The backend can receive a separate X-ray image value and pass it to the AI vision processing component.

Figure 3.3 – Medical Image Processing Workflow



3.15 Health AI Chatbot

The health AI chatbot provides a conversational interface for general health-related questions.

Users can enter questions such as:

- What is a CBC test?
- What is blood pressure?
- What does a laboratory value mean?
- What should I do when I have a fever?

The chatbot sends the user a message to: post /api/chat

The backend then processes the message using the AI service and returns a generated response.

The chatbot is designed to:

- Explain medical terms
- Provide general health information
- Answer laboratory-related questions
- Explain general health concepts
- Provide general lifestyle information
- Recommend professional medical assistance when appropriate

It is not designed to provide medical diagnosis or medication prescriptions.

3.16 Multilingual Support

Health AI supports two languages:

1. English
2. Tamil

The selected language is sent to the backend along with the user's request.

Table 3.2 – Language Processing

Language	Response
English	Simple English explanation
Tamil	Simple Tamil explanation with important medical terms retained where useful

For medical reports, numerical values, units, and reference ranges should remain consistent with the original report even when the explanatory text is translated.
This prevents translation from unintentionally changing the meaning of laboratory information.

3.17 React Markdown

React Markdown is used to display AI-generated responses in a structured format.
The AI response can contain:

- Headings
- Paragraphs
- Lists
- Bold text
- Tables
- Separators

React Markdown converts the generated Markdown into formatted HTML elements within the application interface.
This improves the readability of AI-generated medical explanations.

3.18 PDF Processing

PDF support is included in the health AI application for both input and output.

Input

The application accepts medical reports in PDF format.
The PDF processing module extracts available textual information from the uploaded document before sending it for AI analysis.

Output

The application provides a downloadable PDF containing the generated analysis.
The PDF output allows users to preserve the result of a report analysis for later reference.

3.19 Local Storage

Browser local storage is used to maintain report history in the current application implementation.

After a report is successfully analysed, information such as:

- Report ID
- Report type
- Analysis
- Date
- File name
- Selected language

can be stored as a history record.

This allows previously analysed reports to be displayed through the History module.

CHAPTER 4

SYSTEM DESIGN AND WORKING PRINCIPLE

4.1 Introduction

The health AI system is designed as a web-based Artificial Intelligence application that assists users in understanding medical reports and obtaining general health information. The system combines several technologies into a single application, including:

- User authentication
- Medical report upload
- OCR-based text extraction
- Image preprocessing
- AI-based report analysis
- Medical image observation
- Health AI chatbot
- English and Tamil language support
- Report history
- PDF report generation

The system follows a modular architecture in which each functionality performs a specific task while communicating with the other modules through the application and backend APIs.

The overall design is intended to provide a simple workflow for the user:

Login → Upload Report → Extract Information → AI Analysis → View Result → Save / Download

4.2 Overall System Architecture

The HealthAI system consists of three major layers:

4.2.1 Presentation Layer

The presentation layer is responsible for the user interface.

It is developed using React.js and provides pages such as:

- Home
- Login
- Reports
- Chat
- History
- Profile

The user interacts with the application through buttons, forms, file-upload components, language selection, report preview, and analysis-result sections.

4.2.2 Application Layer

The application layer handles the main business logic of HealthAI.

It manages:

- User authentication state
- File processing
- OCR execution
- API requests
- Report analysis requests
- Chat requests
- History management
- PDF generation

The frontend communicates with the backend through HTTP requests.

4.2.3 AI Processing Layer

The AI processing layer performs the intelligent processing required by the application.

It is responsible for:

- Medical report explanation
- General health question answering
- Image-based observation
- Multilingual response generation

The backend communicates with the configured Groq AI service for these operations.

Figure 4.1 – Overall HealthAI System Architecture

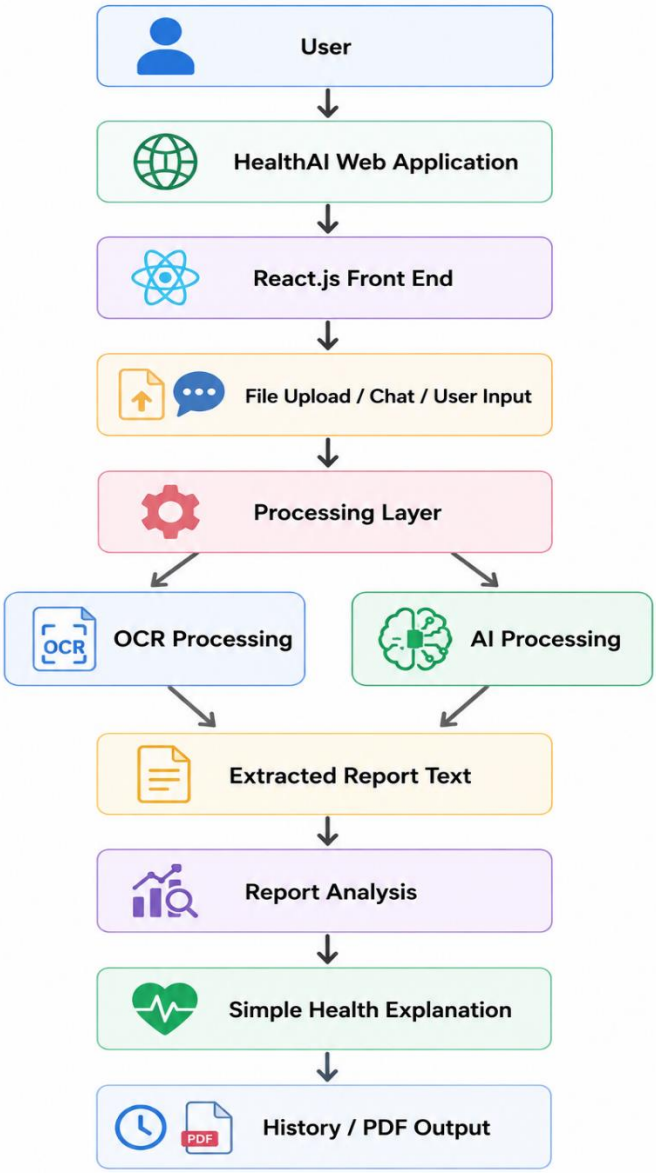
4.3 Overall System Workflow

The complete working process of HealthAI can be divided into the following stages:

1. User authentication
2. Report upload
3. File identification
4. Image/PDF processing
5. OCR text extraction
6. Text normalization
7. AI-based analysis
8. Medical image observation, when applicable
9. Result presentation
10. History storage
11. PDF generation

The workflow allows the user to complete the report-analysis process without manually transferring information between multiple applications.

Figure 4.1– Overall HealthAI Workflow



4.3 User Interaction Flow

The user interaction begins after successful authentication.

The user enters the HealthAI application and selects the required functionality. The main interaction paths are:

Path 1 – Medical Report

User → Reports → Upload Report → Analyse → View Result

Path 2 – Health Chat

User → Chat → Enter Question → Send → View Response

Path 3 – Previous Reports

User → History → Select Previous Report → View Result

Path 4 – Download

User → Analysis Result → Download PDF

This structure provides a single application interface for the major functions of the system.

4.4 Authentication Flow

Authentication is the first controlled stage of the application.
The system checks whether a valid authenticated user is available before allowing access to protected application pages.
The implemented authentication flow is:

User
↓
Login Page
↓
Firebase Authentication
↓
Authentication Verification
↓
Authenticated User
↓
HealthAI Application

The project currently uses Google-based authentication for user login. The authentication workflow already prepared for the project shows the user moving from the login page through Firebase verification before entering the HealthAI application.
If authentication is unsuccessful, the user remains outside the protected application area.

4.5 Medical Report Processing Flow

The medical report workflow begins when an authenticated user selects a report.
The system first identifies the uploaded document and determines the appropriate processing path.
The workflow is:

Report Upload
↓
File Identification
↓
PDF / Image Processing
↓
Information Extraction
↓
Report Analysis
↓
Result Display

The system does not require the user to manually copy the report information into another application.

4.6 Report Input and Processing Decision

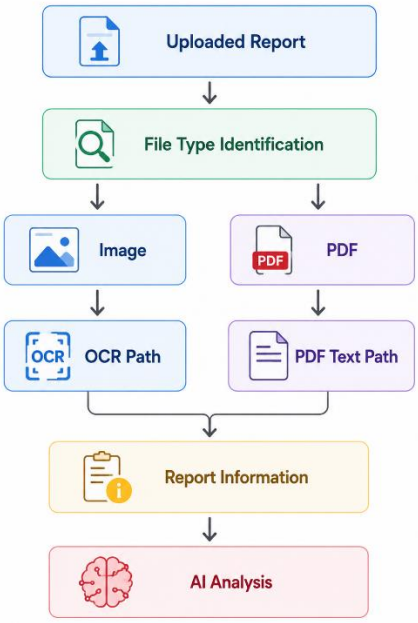
After a report is uploaded, the application determines whether the input is an image or a PDF.

Image Input -For an image-based report, the image is passed through the OCR processing path.

PDF Input -For a PDF report, the available textual information is extracted through the PDF-processing path.

Both paths eventually provide information to the report-analysis stage.

Figure 4.2 – Report Input Processing Flow



4.7 OCR and Extracted Text Flow

For image-based reports, the extracted information becomes the intermediate data between the uploaded document and AI analysis.

The processing sequence is:

Medical Report Image

↓

Pre-processed Image

↓

OCR Output

↓

Text Normalization

↓

Extracted Report Text

↓

AI Analysis

The important design principle here is that OCR output is treated as extracted information rather than automatically assumed to be completely accurate.
If the original image contains unclear characters or values, the resulting information may also be uncertain.
Therefore, the AI analysis stage must preserve uncertainty instead of silently replacing unclear information.

4.8 Report Analysis Data Flow

Once report information is available, the frontend sends the required information to the backend.
The backend receives the report data and prepares the request for AI processing.
The data flow is:

React Front End
↓
Analysis Request
↓
Express Backend
↓
Report Data Preparation
↓
AI Service
↓
Analysis Response
↓
React Front End
↓
Result Display

The current backend design accepts report text, optional report image, optional X-ray image, and language selection as analysis inputs.

4.9 AI Analysis Decision Flow

The report-analysis process is designed around the information actually available in the uploaded report.
The analysis flow can be represented as:

Report Information
↓
Is the value available?
↙ ↘
Yes **No**
↓ ↓
Process Available **Mark as Not Available / Unclear**

↓

Generate Explanation

↓

Display Result

This design prevents the system from filling missing information with unsupported assumptions.

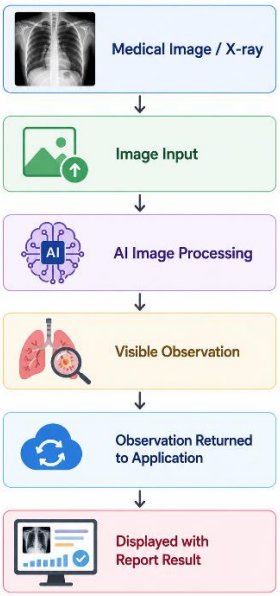
Similarly, when OCR produces an uncertain value, the system should indicate that the value is unclear rather than presenting an invented correction.

4.10 Medical Image and X-ray Flow

When a medical image or X-ray is included with the report, it follows a separate image-analysis path.

The image is provided to the image-processing component and the resulting information is combined with the report-analysis workflow.

Figure 4.3 – Medical Image / X-ray Data Flow



The purpose of this module is limited to providing observations from the available image. It should not be represented as a confirmed clinical diagnosis.

4.11 HealthAI Chat Flow

The chatbot follows a separate request-response flow.

The user enters a general health question through the Chat page.

The system then performs the following sequence:

User Question

↓

Chat Interface

↓

Chat API Request

↓

Backend

↓

AI Processing

↓

Generated Response

↓

Chat Interface

The current implementation uses the /api/chat endpoint for this communication.
The response is returned to the frontend and displayed in a readable conversational format.

4.12 Language Selection Flow

The language selected by the user becomes part of the request sent to the backend.
The flow is:

User Selects Language

↓

Language Value

↓

Frontend Request

↓

Backend

↓

AI Response Generation

↓

Selected Language Response

The current application supports:

- English
- Tamil

The language mechanism is applied to both report analysis and chatbot interaction.
For report analysis, numerical values and units should remain consistent with the original report while the explanatory content is generated in the selected language.

4.13 Result Presentation Flow

After the AI processing is completed, the generated response is returned to the frontend.
The frontend converts the response into a readable interface.
The presentation flow is:

AI Response

↓

Frontend Receives Result

↓

Markdown / Structured Formatting

↓

Report Sections

↓

User View

The result is therefore presented as a structured report rather than as raw backend text. This improves readability and allows important findings to be visually separated.

4.14 Report History Flow

After a report has been successfully analysed, the result can be maintained as part of the user's report history.

The logical flow is:

Completed Analysis

↓

Create History Record

↓

Store Report Information

↓

History Page

↓

User Selects Previous Report

↓

View Previous Analysis

The history feature allows users to access previous results without repeating the complete analysis process.

4.15 PDF Output Flow

The analysis result can also be converted into a downloadable PDF.

The process is:

Completed Analysis

↓

Formatted Analysis

↓

PDF Generation

↓

PDF File

↓

User Download

The PDF acts as a persistent copy of the generated analysis and can be retained by the user for reference.

4.16 Frontend–Backend Communication

The frontend and backend communicate through HTTP requests.

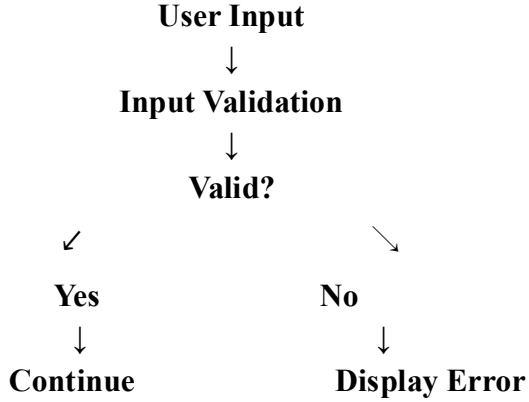
The major communication paths are:

Report Analysis
Frontend → /api/analyze → Backend → AI Service → Backend → Frontend
Chat
Frontend → /api/chat → Backend → AI Service → Backend → Frontend

The backend therefore acts as the intermediate processing layer between the React application and the external AI service.
This architecture also keeps sensitive backend configuration, such as the AI API credential, outside the frontend application.

4.17 Error and Validation Flow

The system includes validation before major processing operations.
The validation flow is:



Examples of conditions requiring validation include:

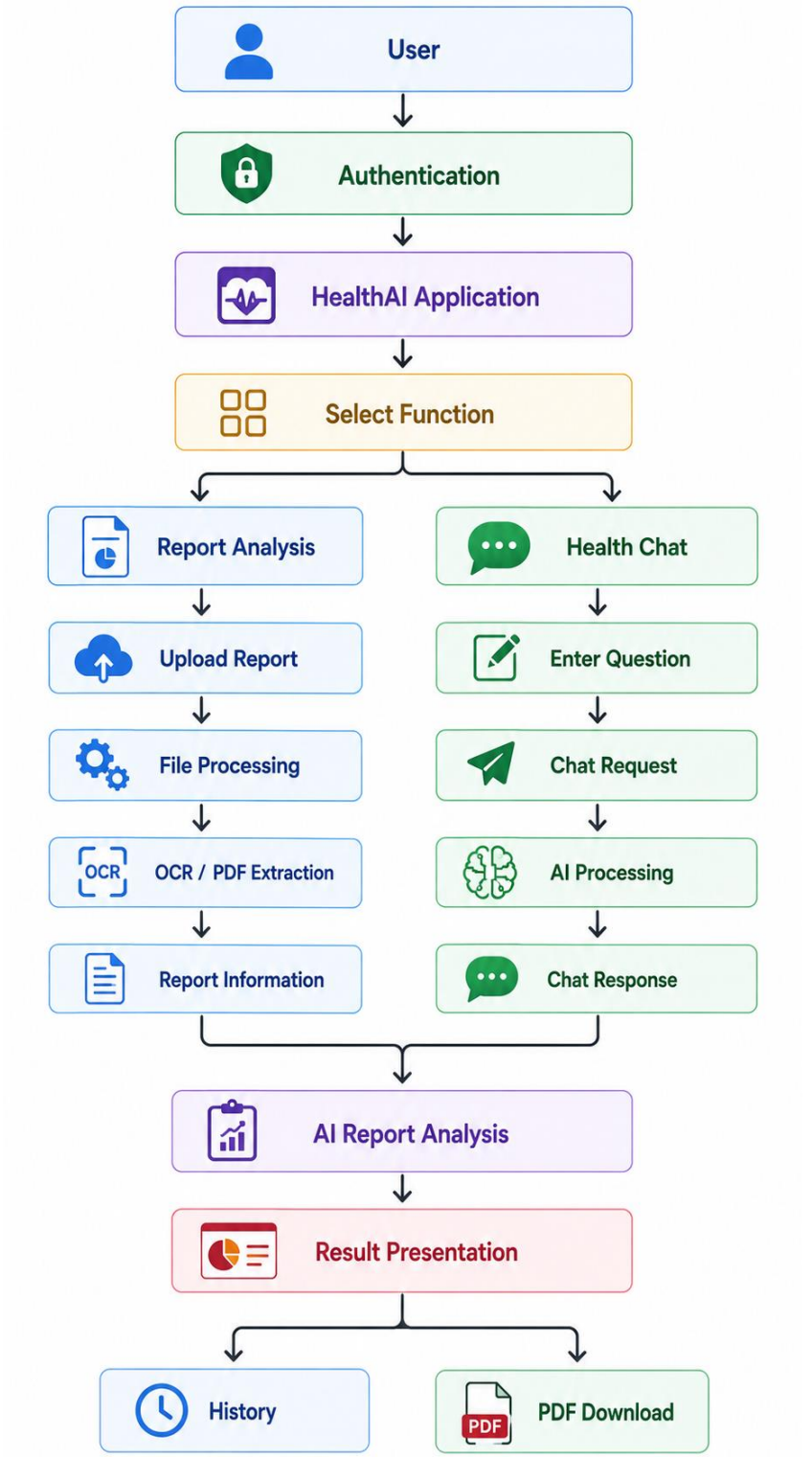
- Empty report
- Empty chat message
- Unsupported input
- Failed OCR processing
- Failed backend request
- Empty AI response
- Authentication failure
- Network failure

The purpose of this design is to prevent invalid input from continuing through the complete processing pipeline.

4.18 Complete End-to-End System Flow

The complete HealthAI workflow can be represented as follows:

Figure 4.4 – Complete HealthAI Workflow



This represents the complete interaction between the major modules without repeating the individual technology descriptions already presented in Chapter 3.

4.19 Module Interaction

The major modules do not operate independently. They exchange information through the application workflow

.

Table 4.1 – HealthAI Module Interaction

Source Module	Destination Module	Information Exchanged
Authentication	Application	User authentication state
Report Upload	File Processing	Selected report
File Processing	OCR / PDF Processing	Report document
OCR / PDF Processing	AI Analysis	Extracted report information
Image Input	Image Processing	Medical image / X-ray
AI Analysis	Result Display	Generated explanation
Chat Interface	Chat API	User question
Chat API	Result Display	AI response
Result Display	History	Completed analysis
Result Display	PDF Module	Formatted analysis

4.20 System Design Summary

The HealthAI system is designed around a connected information flow rather than isolated functionalities.

The central design principle is:

Input → Processing → AI Interpretation → Structured Output

Different inputs follow different processing paths, but they ultimately converge at the result-generation stage.

The report workflow processes uploaded documents, the chatbot workflow processes user questions, and the image workflow processes medical images or X-rays.

The resulting information is presented through the same HealthAI interface and can be maintained through history or converted into a PDF.

4.21 Chapter Summary

This chapter presented the **system design and architectural workflow** of HealthAI.

The chapter focused on how the modules communicate and how information moves through the application, without repeating the individual technology descriptions

already covered in Chapter 3. The design includes authentication flow, report processing flow, OCR-to-analysis flow, image observation flow, chatbot communication, multilingual processing, history management, PDF generation, frontend-backend communication, and error validation.

The next chapter can therefore focus specifically on **System Implementation**, where the actual source-code structure, React components, backend routes, OCR implementation, authentication implementation, and AI integration can be documented.

CHAPTER 5

SYSTEM IMPLEMENTATION

5.1 Introduction

This chapter describes the practical implementation of the HealthAI system. The implementation follows the architecture and workflows described in Chapter 4.

The application is implemented as a web-based system consisting of a React frontend and a Node.js/Express backend. The frontend manages user interaction and application pages, while the backend manages API requests and communication with the AI service.

The major implemented modules are:

- User authentication
- Home interface
- Medical report upload
- OCR processing
- Medical report analysis
- Medical image/X-ray analysis
- HealthAI chatbot
- English and Tamil language support
- Report history
- PDF report generation

5.2 Application Structure

The project is organized into separate frontend and backend components.

Frontend

The frontend contains the user-facing pages and reusable application components.

The major pages include:

Page	Function
Login	User authentication
Register	Account-related interface
Home	Main application dashboard
Reports	Medical report upload and analysis
Chat	HealthAI chatbot
History	Previous report results
Profile	User information

Backend

The backend contains the server-side API implementation.

The major endpoints are:

Endpoint	Function
/	Server status
/api/analyze	Medical report analysis
/api/chat	HealthAI chatbot

5.3 Frontend Implementation

The frontend is implemented using React components.

Each major function is maintained as a separate page or component. This provides a modular structure and makes individual functions easier to maintain.

The main application routing controls access to:

- Home
- Reports
- Chat
- History
- Profile
- Login
- Register

Protected pages are displayed only when the user has been authenticated.

5.4 Authentication Implementation

Firebase Authentication is integrated into the application for user login.

The application monitors the authentication state and updates the current user information accordingly.

The authentication process is implemented using Firebase authentication methods and Google sign-in.

After successful authentication, the user is redirected to the HealthAI home page.

If the authentication state is not available during application startup, a loading screen is displayed before the application determines the user's access status.

5.5 Medical Report Upload Implementation

The Reports page provides the interface for uploading medical documents.

The implementation allows the user to:

1. Select a report.
2. Display the selected file information.
3. Preview the report when supported.
4. Select the report type.
5. Start the analysis process.
6. View the generated result.

The uploaded file is passed to the appropriate processing function depending on its format.

The interface also displays a processing state while the analysis is being performed.

5.6 Image Preprocessing Implementation

Image preprocessing is implemented before OCR execution.

The preprocessing function performs the following operations:

1. Loads the selected image.
2. Creates a larger canvas.
3. Scales the image.
4. Creates a white background.
5. Converts the image into grayscale.
6. Enhances contrast.
7. Converts the processed image into PNG format.

This produces a processed image that is then supplied to the OCR module.

The preprocessing implementation is intended to improve recognition of printed medical report text.

5.7 OCR Implementation

Tesseract.js is used to extract text from image-based medical reports.

The implementation performs OCR after preprocessing the image.

The extracted text is then normalized before being passed to the AI analysis process.

The normalization process includes:

- Standardizing comma characters.
- Converting decimal comma notation where applicable.
- Removing unnecessary spaces.
- Removing excessive blank lines.
- Preserving the extracted medical information.

The system does not intentionally replace unclear medical values with guessed values.

5.8 Medical Report Analysis Implementation

After OCR or PDF processing, the extracted report information is sent to the backend.

The frontend sends the analysis request to:

POST /api/analyze

The request can contain:

- Report text
- Report image
- X-ray image
- Selected language

The backend validates the report information before continuing.

If no report text is available, the backend returns an error indicating that the report text is empty.

If valid information is available, the request is passed to the AI-processing stage.

5.9 Backend Implementation

The backend is implemented using Express.js.

The server performs the following tasks:

- Accepts frontend requests.
- Validates incoming data.
- Processes report requests.
- Processes chatbot requests.
- Communicates with the Groq AI service.
- Returns generated responses.
- Handles processing errors.

The server also enables CORS and JSON request processing.
The application server runs on port **5000** during development.

5.10 Groq AI Integration

The Groq SDK is initialized in the backend using an API key stored in the environment configuration.
The AI model configured for the application is:
openai/gpt-oss-120b
The backend creates a chat-completion request containing:

- System instructions
- User-provided information
- Language instruction
- Report information where applicable

The generated response is then extracted from the AI response and returned to the frontend.

5.11 Medical Report Analysis Prompt

The report-analysis implementation uses a controlled system instruction to guide the AI response.
The instructions require the system to:

- Use only available report information.
- Avoid inventing values.
- Avoid unsupported diagnosis.
- Identify unclear information.
- Provide general health information.
- Avoid prescribing medication.
- Recommend professional consultation when appropriate.

The analysis response is returned to the frontend as structured text.
This controlled prompt is important because the AI is being used to explain medical information rather than to act as an independent medical decision-maker.

5.12 Medical Image and X-ray Implementation

The backend supports optional medical image and X-ray inputs.
When an image is available, it can be passed to the AI image-processing component together with the report information.
The image-processing instructions require the system to focus on visible observations.

The implementation avoids presenting the output as a confirmed diagnosis.
The system also allows the AI to indicate when the image quality is insufficient for reliable observation.

5.13 HealthAI Chatbot Implementation

The chatbot is implemented through the Chat page.
The user enters a question into the chat input field.
When the user presses the Send button, the frontend sends the request to:
POST /api/chat
The request contains:

- User message
- Selected language

The backend validates the message and sends it to the AI service.
The response is then returned to the frontend.

Chat Implementation Flow

User Message
↓
Chat Component
↓
Fetch API Request
↓
/api/chat
↓
Express Backend
↓
Groq AI
↓
Generated Reply
↓
Chat Component

5.14 Chat Interface Implementation

The Chat component maintains two major states:

- messages
- loading

The messages state stores the conversation displayed on the page.
Each message contains:

- Role
- Content

The role identifies whether the message was generated by the user or HealthAI.
The loading state is used to display an intermediate message while the AI is processing the request.
This prevents the user from sending multiple requests simultaneously.

5.15 Markdown Response Rendering

AI-generated responses are displayed using React Markdown.

This allows the application to render:

- Headings
- Paragraphs
- Lists
- Bold text
- Tables

The implementation applies custom styling to these elements so that AI responses remain consistent with the HealthAI interface.

This is particularly useful for medical explanations because information can be separated into readable sections.

5.16 Multilingual Implementation

The application supports both English and Tamil.

The selected language is passed from the frontend to the backend.

The backend creates a language-specific instruction for the AI service.

For English, the system generates a simple English response.

For Tamil, the system generates a simple Tamil response while retaining important medical terminology in English where appropriate.

This functionality is available for both:

- Medical report analysis
- HealthAI chatbot

5.17 Report History Implementation

After an analysis is completed, the result can be stored as a history entry.

The history information can contain:

- File name
- Report type
- Analysis result
- Date
- Language

The History page displays previously processed reports and allows the user to access their stored results.

This avoids requiring the user to repeat the analysis process for previously processed documents.

5.18 PDF Report Implementation

The generated analysis can be converted into a downloadable PDF.

The PDF-generation process takes the completed analysis and formats it into a document.

The resulting document can be downloaded by the user.

The PDF feature provides a convenient way to retain the generated report outside the application.

5.19 Error Handling Implementation

Error handling is implemented on both frontend and backend sides.

Frontend Errors

The frontend handles conditions such as:

- Empty input
- Failed API request
- AI connection failure
- OCR failure
- Authentication failure

Backend Errors

The backend handles:

- Missing report text
- Missing chatbot message
- AI service errors
- Empty AI responses
- Unexpected server errors

The backend returns JSON responses containing a success status and error information where appropriate.

5.20 Loading State Implementation

Loading states are implemented during operations that may require processing time.

For report analysis, the interface indicates that processing is in progress.

For chatbot requests, the interface displays:

"HealthAI is thinking..."

The Send button is disabled while the chatbot request is being processed.

This prevents duplicate requests and gives the user clear feedback that the system is working.

5.21 Frontend–Backend Integration

The frontend communicates with the backend using the JavaScript fetch() API.

For medical reports:

Frontend

↓

POST /api/analyze

↓

Express Backend

↓

Groq AI

↓

Analysis Response

↓

Frontend

For chatbot requests:

Frontend

↓

POST /api/chat

↓

Express Backend

↓

Groq AI

↓

Chat Response

↓

Frontend

This integration separates user-interface responsibilities from server-side AI processing.

5.22 Environment Configuration

The Groq API credential is maintained through an environment variable.

The backend uses:

GROQ_API_KEY

The key is not directly written into the frontend code.

The server checks whether the environment variable is available when it starts.

If the key is missing, the backend displays an error message in the server console.

5.23 Implementation Summary

The HealthAI implementation combines the modules described in the system design into a functional web application.

The implementation includes:

- React-based user interface
- Firebase authentication
- Medical report upload
- OCR preprocessing and extraction
- Backend API processing
- Groq AI integration
- Medical image observation
- HealthAI chatbot
- English and Tamil responses
- Report history
- PDF generation
- Error handling
- Loading-state management

The implementation provides the technical foundation required for the testing and result evaluation presented in the following chapter.

CHAPTER 6

RESULTS AND SYSTEM OUTPUTS

6.1 Introduction

This chapter presents the results obtained from the implemented HealthAI application. The results demonstrate the working of the major modules developed in the system, including user authentication, medical report processing, OCR-based extraction, AI report analysis, medical image observation, chatbot interaction, multilingual support, report history, and PDF generation.

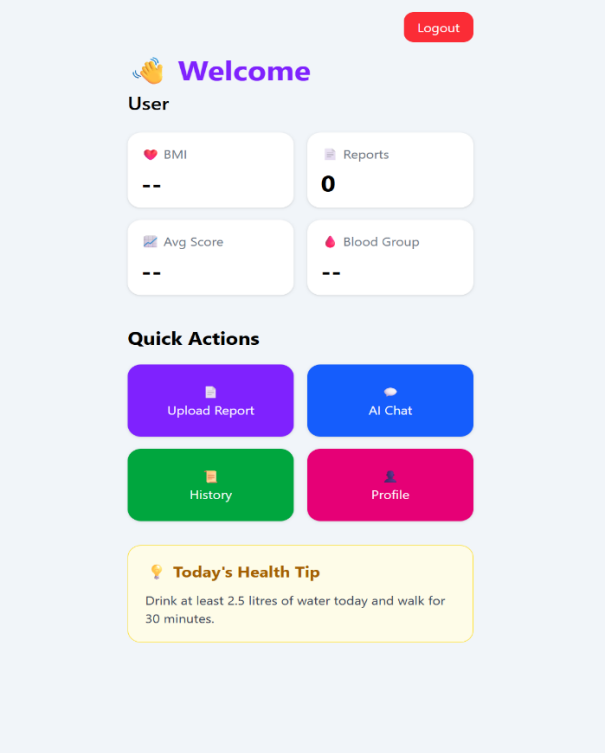
The evaluation focuses on the visible behaviour of the application and the successful completion of the intended workflows.

6.2 HealthAI Home Page

The Home page provides the main entry point after successful authentication.

It provides access to the major functions of the application and allows the user to select the required healthcare service.

Figure 6.1 – HealthAI Home Page



Observation:

The Home page provides a centralized interface from which the user can access report analysis, health chat, history, and other available application features.

6.3 User Authentication Result

The authentication module was tested by accessing the application through the login interface.

After successful Google authentication, the user is redirected to the main HealthAI application.

Figure 6.2 – HealthAI Login Page

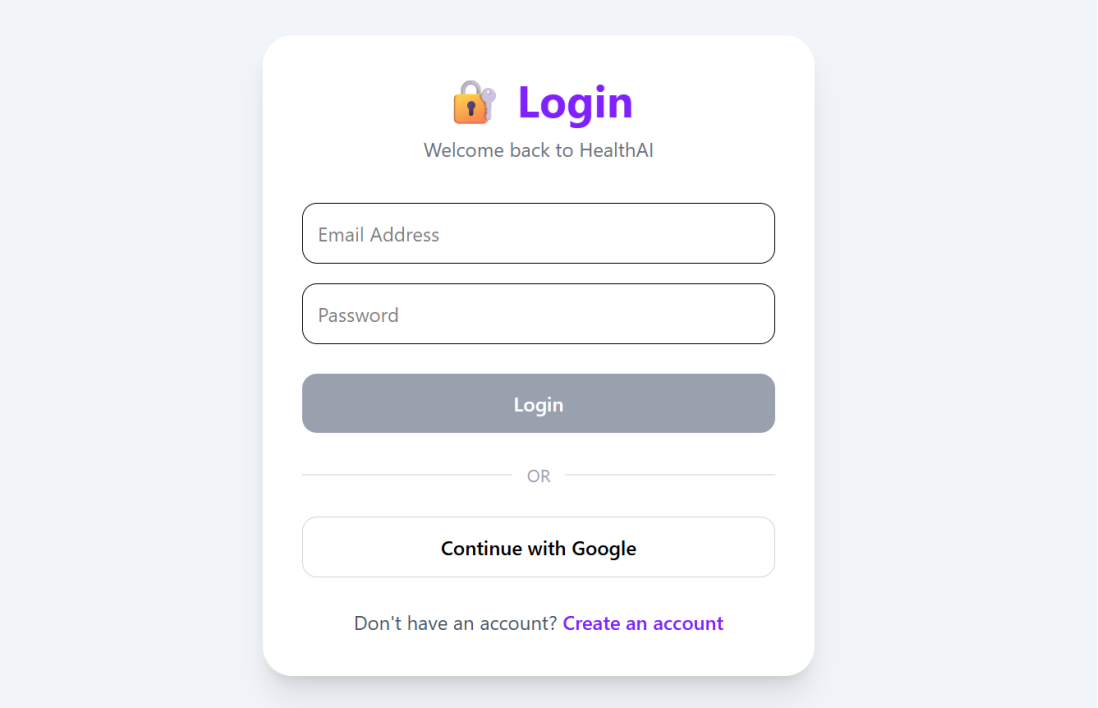
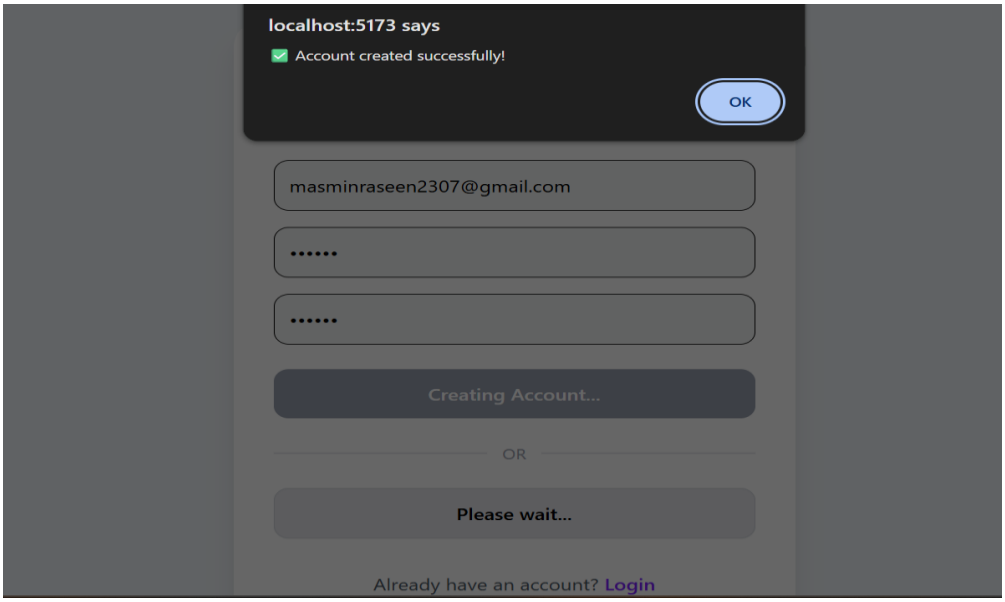


Figure 6.3 – Successful Google Authentication



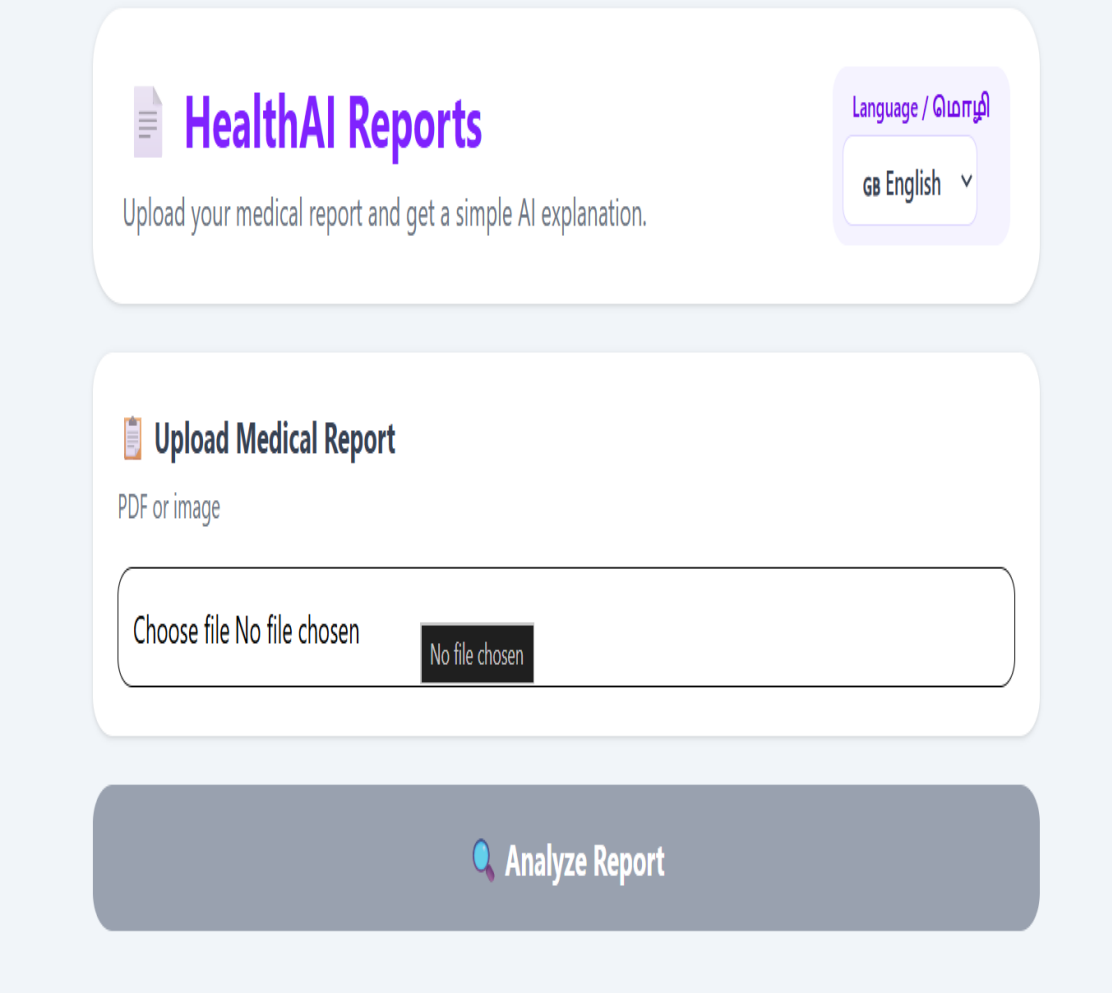
Result:

The authentication workflow successfully controls access to the protected application pages.

6.4 Medical Report Upload Result

The Reports module allows the user to select a medical report for processing.
After selecting the file, the application displays information about the selected report and provides a preview where supported.

Figure 6.4 – Medical Report Upload Interface



The interface displays:

- Selected file
- File size
- Report preview
- Report type
- Analyse button

Result:

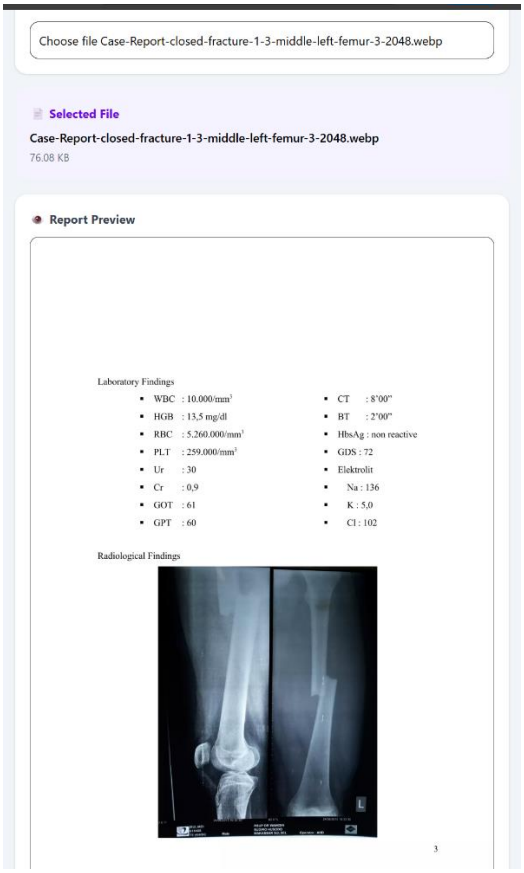
The report-upload workflow provides the user with a clear confirmation of the selected document before analysis.

6.5 OCR Processing Result

The OCR module was used to extract textual information from image-based medical reports.

The processing sequence consists of image preprocessing followed by OCR extraction and text normalization.

Figure 6.5 – OCR Processing Output



The extracted text is subsequently used as input for AI-based report analysis.

Result:

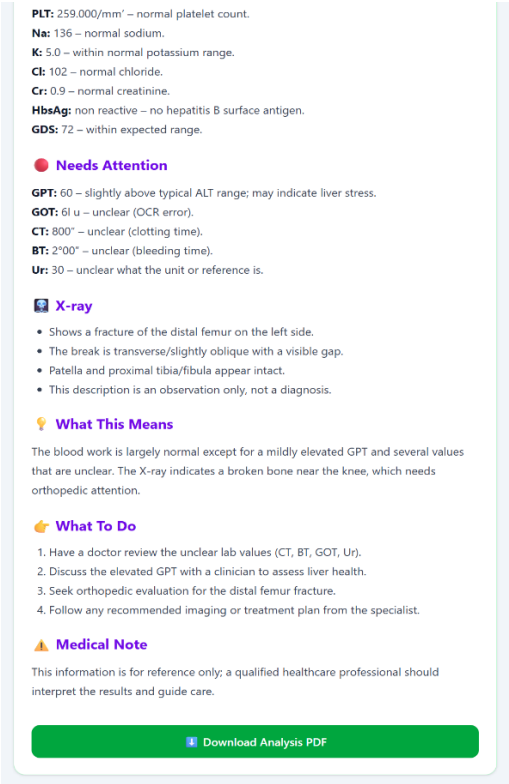
The OCR module successfully provides machine-readable text from supported report images. However, the accuracy of extracted values depends on the quality and clarity of the original image.

6.6 Medical Report Analysis Result

After the report is processed, the extracted information is sent to the AI analysis module.

The generated result is presented in a structured format instead of displaying raw extracted text.

Figure 6.6 – AI Medical Report Analysis



The result can contain sections such as:

- Overall status
- Important findings
- Normal findings
- Values requiring attention
- General explanation
- Medical note

The exact content depends on the information available in the uploaded report.

Result:

The application converts extracted medical information into a simpler user-readable explanation.

6.7 Handling of Unclear Values

One important result of the implemented system is its handling of uncertain information.

OCR may produce unclear characters or values when the uploaded report is blurred, damaged, poorly scanned, or contains complex formatting.

Instead of automatically replacing such information with a guessed value, the system can indicate that the value is unclear or requires verification.

6.8 Medical Image and X-ray Result

The system also supports medical image and X-ray observation.

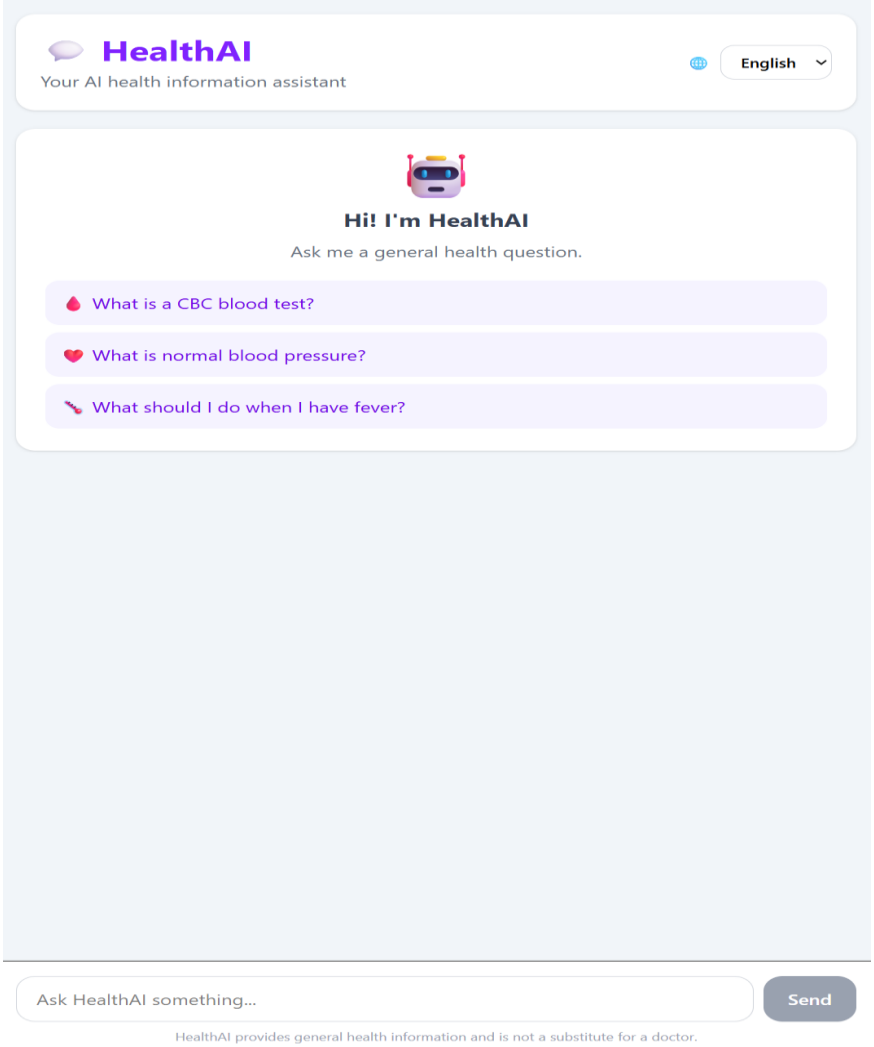
When an image is provided, the image-processing component produces an observation based on the visible information.

6.9 HealthAI Chatbot Result

The Chat module was tested by entering general health-related questions.

The chatbot processes the question and displays the generated response within the conversation interface.

Figure 6.9 – HealthAI Chat Interface



The interface displays:

- User message
- HealthAI response
- Loading state
- Message history
- Input field
- Send button

Result:

The chatbot successfully provides a conversational interface for general health information.

6.10 Sample Chat Interaction

A sample interaction can be presented to demonstrate the chatbot workflow.

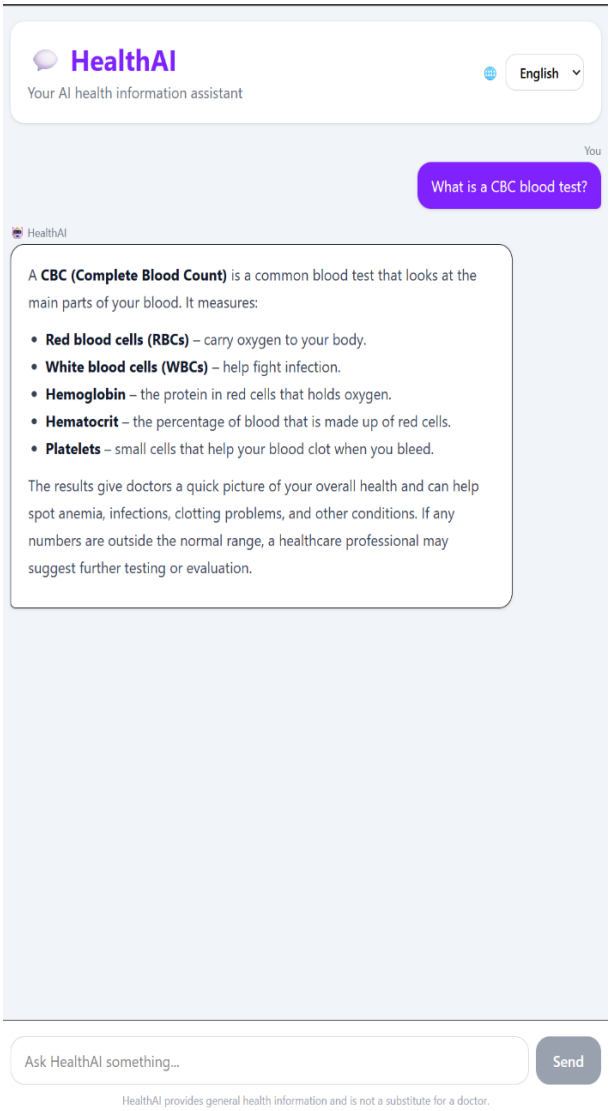
User Question

"What is a CBC blood test?"

HealthAI Response

The system provides a short explanation of the purpose of a CBC test and explains the major parameters in simple language.

Figure 6.10 – Sample HealthAI Chat Response



Result:

The chatbot converts a general health question into a user-readable response without requiring the user to navigate to a separate information source.

6.11 Tamil Language Output

The multilingual feature was implemented to allow users to interact with HealthAI in Tamil as well as English.

When Tamil is selected, the AI response is generated in Tamil while important medical terminology and numerical values can be retained where necessary.

Figure 6.11 – Tamil HealthAI Response

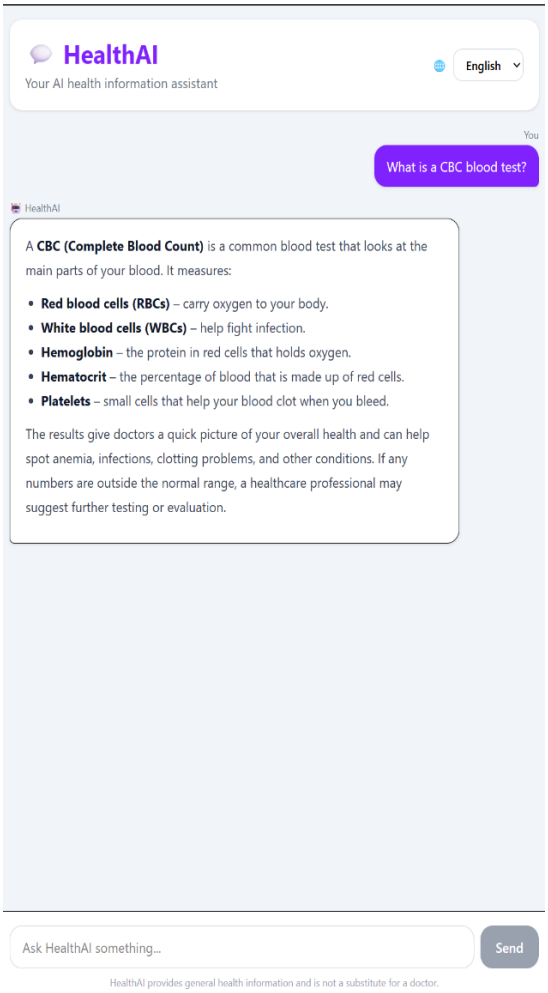


Result:
The multilingual interface improves accessibility for users who prefer Tamil for general health information.

6.12 English Language Output

English is provided as the default language option for the application.

Figure 6.12 – English HealthAI Response



The English response is designed to use simple wording so that technical medical information is easier to understand.

6.13 Report History Result

The History module provides access to previously analysed reports.
The history interface can display information such as:

- Report name
- Report type
- Analysis date
- Previous analysis

Result:

Users can access previously processed report results without performing the complete analysis process again.

6.14 PDF Download Result

The completed analysis can be converted into a downloadable PDF.

The generated document provides a permanent digital copy of the analysis result.

Result:

The PDF functionality provides a convenient method for retaining and sharing the generated information.

6.15 Functional Results

The major functional modules were evaluated based on whether the intended operation could be completed.

Table 6.1 – Functional Test Summary

S. No.	Function	Expected Result	Status
1	User Authentication	User successfully enters the application	Pass
2	Report Upload	Selected report is displayed	Pass
3	OCR Processing	Text is extracted from supported images	Pass
4	Report Analysis	AI explanation is generated	Pass
5	Image/X-ray Processing	Observation is generated when supported	Pass
6	Health Chat	AI response is displayed	Pass
7	English Response	English response is generated	Pass
8	Tamil Response	Tamil response is generated	Pass
9	Report History	Previous analysis can be accessed	Pass
10	PDF Generation	Analysis can be downloaded	Pass

6.16 Output Quality Observation

The output quality depends on the quality of the input provided to the system.

For text-based reports and clear report images, the extracted information can be processed more reliably.

For low-quality images, OCR may produce uncertain characters or values. The system therefore emphasizes the identification of unclear information instead of automatically assuming that every extracted value is correct.

Similarly, medical image observations depend on the quality and visibility of the uploaded image.

Therefore, the output should be considered an AI-assisted explanation of the provided information and not a replacement for professional medical interpretation.

6.17 User Interface Observation

The implemented interface provides a consistent layout across the major modules.

The application uses:

- Card-based layouts
- Clear headings
- Separate user and AI messages
- Structured report sections
- Responsive controls
- Visible loading states
- File preview
- Action buttons

The chatbot uses separate message styles for the user and HealthAI, making the conversation easy to follow.

6.18 Overall System Result

The completed HealthAI application integrates the major project requirements into a single platform.

The final system provides:

Authentication

↓

Medical Report Upload

↓

OCR / PDF Processing

↓

AI Analysis

↓

Medical Image Observation

↓

HealthAI Chat

↓

English / Tamil Support

↓

History

↓

PDF Output

The implementation demonstrates that the proposed workflow can be performed through a unified web application rather than requiring separate tools for each operation.

6.19 Chapter Summary

This chapter presented the results and visible outputs of the implemented HealthAI system.

The results demonstrate the functioning of the major modules, including authentication, medical report upload, OCR processing, AI-based report analysis, medical image observation, chatbot interaction, multilingual support, report history, and PDF generation.

The results also demonstrate the importance of input quality and uncertainty handling when processing medical documents and images.

The next chapter will focus on **Testing and Performance Analysis**, where the individual modules will be evaluated systematically using test cases, expected results, actual results, and observations.

CHAPTER 7
TESTING AND PERFORMANCE ANALYSIS

7.1 Introduction

Testing is an important stage in the development of the HealthAI system. It is performed to verify whether the implemented modules operate according to their intended functionality.

The testing process covers the major user and system operations, including authentication, report upload, OCR processing, AI analysis, chatbot interaction, language selection, history management, and PDF generation.

The objective of testing is to identify functional errors, input-validation issues, communication failures, and unexpected system behaviour before considering the application complete.

7.2 Testing Methodology

The HealthAI system was tested using functional test cases.

Each test case contains:

- Test Case ID
- Module
- Test Condition
- Expected Result
- Actual Result
- Status

The testing approach focuses primarily on **black-box functional testing**, where the system behaviour is evaluated from the user's perspective without depending on the internal implementation details.

7.3 Functional Testing

Functional testing verifies whether each major feature performs its intended operation.

Table 7.1 – Functional Test Cases

Test ID	Module	Test Condition	Expected Result	Status
TC01	Authentication	Open login page	Login interface displayed	Pass
TC02	Authentication	Complete Google login	User enters application	Pass
TC03	Report Upload	Select valid report	File information displayed	Pass
TC04	Report Upload	Select report image	Preview displayed	Pass

TC05	OCR	Upload readable image	Text extracted	Pass
TC06	Report Analysis	Submit valid report	AI analysis generated	Pass
TC07	Chat	Enter valid question	AI response displayed	Pass
TC08	Tamil Language	Select Tamil	Tamil response generated	Pass
TC09	History	Open History page	Previous result displayed	Pass
TC10	PDF	Select download option	PDF generated/downloaded	Pass

7.4 Authentication Testing

Authentication testing verifies whether access to the application is correctly controlled.

Table 7.2 – Authentication Test Cases

Test ID	Test Condition	Expected Result	Status
AT01	Open application without authentication	User is directed to Login	Pass
AT02	Select Google login	Google authentication interface opens	Pass
AT03	Complete successful authentication	User is redirected to Home	Pass
AT04	Access protected page after login	Page opens successfully	Pass
AT05	Sign out and access protected page	User is redirected to Login	Pass

The results indicate that the authentication mechanism correctly separates authenticated and unauthenticated access.

7.5 Medical Report Upload Testing

The report-upload module was tested using supported medical document formats.

Table 7.3 – Report Upload Test Cases

Test ID	Test Condition	Expected Result	Status
RT01	Upload image report	Image accepted	Pass
RT02	Upload PDF report	PDF accepted	Pass
RT03	Select report	File name displayed	Pass
RT04	Select image	Preview displayed	Pass
RT05	Start analysis without report	Validation message displayed	Pass

The upload interface provides feedback to the user before the analysis process begins.

7.6 OCR Testing

OCR testing evaluates the extraction of text from medical report images.

Table 7.4 – OCR Test Cases

Test ID	Test Condition	Expected Result	Status
OCR01	Clear printed report	Text extracted correctly	Pass
OCR02	Report with numerical values	Values extracted	Pass
OCR03	Report with decimal values	Decimal formatting retained	Pass
OCR04	Low-quality report image	Unclear values identified for verification	Pass
OCR05	Empty/invalid image	OCR error handled	Pass

OCR performance depends strongly on the quality of the uploaded document.
Clear, high-resolution images provide better extraction results than blurred or low-resolution images.

7.7 AI Report Analysis Testing

The report-analysis module was tested using extracted report information.

Table 7.5 – AI Analysis Test Cases

Test ID	Test Condition	Expected Result	Status
AI01	Submit valid report text	Analysis generated	Pass
AI02	Submit empty report	Error returned	Pass
AI03	Report contains unclear value	Value identified as unclear	Pass
AI04	Report contains normal information	Information explained	Pass

AI05	Report contains abnormal information	Important finding highlighted	Pass
AI06	AI returns empty response	Error handled	Pass

The testing confirms that the report-analysis module can process available report information and return an understandable response.

7.8 Medical Image and X-ray Testing

The image-analysis functionality was tested using supported medical images.

Table 7.6 – Medical Image Test Cases

Test ID	Test Condition	Expected Result	Status
IMG01	Upload readable medical image	Observation generated	Pass
IMG02	Upload X-ray image	Image observation generated	Pass
IMG03	Upload unclear image	Uncertainty indicated	Pass
IMG04	Image not provided	Report analysis continues without image observation	Pass

The image-processing functionality is designed to provide observations rather than present AI output as a confirmed diagnosis.

7.9 Chatbot Testing

The HealthAI chatbot was tested using general health-related questions.

Table 7.7 – Chatbot Test Cases

Test ID	Test Condition	Expected Result	Status
CH01	Enter valid health question	AI response generated	Pass
CH02	Submit empty message	Message is not sent	Pass
CH03	Send question in English	English response generated	Pass
CH04	Send question in Tamil	Tamil response generated	Pass
CH05	Send multiple messages	Messages displayed sequentially	Pass

CH06	AI/server unavailable	Connection error displayed	Pass

7.10 Multilingual Testing

The language functionality was tested using both supported languages.

Table 7.8 – Language Test Cases

Test ID	Language	Test Condition	Expected Result	Status
LAN01	English	Analyse report	English explanation	Pass
LAN02	Tamil	Analyse report	Tamil explanation	Pass
LAN03	English	Ask chatbot question	English response	Pass
LAN04	Tamil	Ask chatbot question	Tamil response	Pass
LAN05	Tamil	Process numerical values	Values preserved	Pass

The results demonstrate that the application can provide responses in the selected language while retaining important numerical information.

7.11 Report History Testing

The History module was tested to ensure that completed analysis results can be accessed later.

Table 7.9 – History Test Cases

Test ID	Test Condition	Expected Result	Status
HIS01	Complete report analysis	Result stored in history	Pass
HIS02	Open History	Previous result displayed	Pass
HIS03	Select previous report	Analysis details displayed	Pass
HIS04	No history available	Empty-history state displayed	Pass

7.12 PDF Generation Testing

The PDF functionality was tested using completed report analyses.

Table 7.10 – PDF Test Cases

Test ID	Test Condition	Expected Result	Status
PDF01	Generate PDF	PDF created	Pass
PDF02	Download PDF	File downloaded	Pass
PDF03	Open generated PDF	Analysis content visible	Pass
PDF04	Generate PDF from different report	Corresponding analysis included	Pass

7.13 Error Handling Testing

Error handling was tested for common invalid conditions.

Table 7.11 – Error Handling Test Cases

Test ID	Error Condition	Expected Behaviour	Status
ERR01	Empty report	Error message displayed	Pass
ERR02	Empty chat message	Request prevented	Pass
ERR03	OCR failure	Processing error displayed	Pass
ERR04	AI service failure	Connection/error message displayed	Pass
ERR05	Empty AI response	Server returns error	Pass
ERR06	Authentication failure	Login error displayed	Pass

The results demonstrate that common failure conditions are handled instead of allowing the application to fail silently.

7.14 Integration Testing

Integration testing verifies communication between connected modules.
The following major integrations were tested:

Authentication Integration

Firebase → Application → Protected Routes → Report Integration
Upload → OCR/PDF Processing → Backend → AI → Result

Chat Integration → Chat UI → Backend → AI → Chat UI

History Integration → Analysis Result → History → Previous Result
PDF Integration→ Analysis Result → PDF Generation → Download

Table 7.12 – Integration Test Summary

Integration	Expected Result	Status
Authentication and Routing	Protected pages accessible after login	Pass
Upload and OCR	Uploaded image processed through OCR	Pass
OCR and AI	Extracted text sent for analysis	Pass
Backend and AI	AI response returned to application	Pass
Chat and AI	Chat response displayed	Pass
Analysis and History	Result stored/accessed	Pass
Analysis and PDF	Result converted to PDF	Pass

7.15 Performance Analysis

The performance of the HealthAI system depends on several factors.

These include:

- Image size
- Image quality
- OCR processing time
- Report complexity
- Network connection
- AI service response time
- Browser performance

OCR processing can take longer for large images because the image may require preprocessing before recognition.

Similarly, AI response time depends on the request size and external AI service response.

Therefore, the application uses loading indicators during operations that may require additional processing time.

7.16 Usability Analysis

The system was designed with usability as an important consideration.

The major usability characteristics include:

- Simple navigation
- Clear page structure
- Visible action buttons
- Report preview
- Separate user and AI messages
- Loading feedback
- Structured analysis results
- Language selection
- History access
- PDF download

These features reduce the number of manual steps required to use the system.

7.17 Reliability Considerations

The system reliability depends on the successful operation of several external and internal components.

The major dependencies include:

- Firebase authentication
- OCR processing
- Backend server
- Groq AI service
- Internet connectivity

The application includes error handling for failures in these areas.

If an external service is temporarily unavailable, the application can display an appropriate error message rather than presenting an incorrect result.

7.18 Testing Limitations

Although the implemented system successfully passes the functional tests described above, several limitations remain.

First, OCR accuracy depends on the quality of the input document. Poor-quality images can result in incorrect or incomplete extraction.

Second, AI-generated information depends on the quality and completeness of the input provided to the model.

Third, medical image observations should not be treated as clinical diagnosis.

Finally, the current testing focuses primarily on functional behaviour of the prototype. A production healthcare system would require broader testing, including security testing, privacy testing, load testing, accessibility testing, clinical validation, and compliance evaluation.

7.19 Overall Testing Result

The functional testing indicates that the major HealthAI modules operate according to their intended workflow.

The following major functions were successfully verified:

- Authentication
- Report upload
- OCR processing
- AI report analysis
- Medical image observation
- Chatbot
- English language response
- Tamil language response
- History
- PDF generation
- Error handling

The testing therefore demonstrates the functional feasibility of the proposed HealthAI application.

7.20 Chapter Summary

This chapter presented the testing methodology and performance observations of the HealthAI system.

Functional, integration, authentication, OCR, AI analysis, chatbot, multilingual, history, PDF, and error-handling tests were considered.

The results indicate that the implemented modules perform their intended functions under the tested conditions. At the same time, limitations related to OCR quality, AI dependency, image quality, network availability, and the need for further production-level validation were identified.

The next chapter will present the **Future Enhancements** of the Health AI system.

CHAPTER 8

FUTURE ENHANCEMENTS

8.1 Introduction

The current Health AI system provides an integrated platform for medical report understanding, general health information, medical image observation, chatbot interaction, multilingual responses, report history, and PDF generation.

However, the system can be further enhanced with additional technologies and features to improve accuracy, usability, security, scalability, and accessibility.

The following enhancements can be considered for future versions of Health AI.

8.2 Improved OCR Accuracy

Future versions can use more advanced document-processing techniques to improve OCR accuracy.

Possible improvements include:

- Automatic document alignment
- Noise removal
- Better image enhancement
- Table-aware OCR
- Handwritten text recognition
- Multi-column report recognition
- Medical terminology correction

These improvements can help reduce errors when processing low-quality or complex medical reports.

8.3 Advanced Medical Report Understanding

The current system primarily explains information available in the uploaded report.

A future version can provide more structured understanding by automatically identifying:

- Test names
- Reported values
- Units
- Reference ranges
- Abnormal indicators
- Report dates
- Patient information
- Laboratory sections

The extracted information could then be displayed in a structured report format.

8.4 Improved Medical Image Analysis

Future versions can improve the medical-image functionality by supporting a wider range of image types.

Possible improvements include:

- Better X-ray image processing
- Multiple medical image formats
- Image-quality assessment
- Region highlighting
- Comparison between previous and current images
- Specialist-oriented visual reporting

Any advanced medical-image feature would require appropriate validation before being used in a clinical environment.

8.5 Longitudinal Health Report Comparison

A future version can compare reports belonging to the same user across different dates. For example:

Previous Report → Current Report → Change Analysis

The system could show changes in available laboratory values over time.
This would help users understand whether a reported value has changed compared with their previous reports.
Such comparisons should be based only on compatible tests, units, and appropriate reference information.

8.6 Enhanced Multilingual Support

The current system supports English and Tamil.
Future versions can extend this capability to additional Indian languages.
Possible languages include:

- Malayalam
- Telugu
- Kannada
- Hindi
- Bengali
- Marathi

The language system could also provide user-selectable interface translations in addition to AI-generated responses.

8.7 Voice-Based Health AI

Voice interaction can be introduced in a future version.
The proposed workflow would be:

User Voice
↓
Speech Recognition
↓
Health AI Processing

↓

Generated Response

↓

Text-to-Speech

↓

Voice Output

This could improve accessibility for users who have difficulty typing or reading long responses.

8.8 Personalized Health Information

Future versions may provide more personalized information by considering user-provided context such as:

- Age group
- Existing medical information
- Previous reports
- Selected health goals
- Relevant lifestyle information

However, such functionality would require strong privacy controls and appropriate safeguards because the application would be processing more sensitive information.

8.9 Improved Chatbot Context

The current chatbot processes individual user questions.

A future version could provide better conversational context so that related questions can be understood across multiple messages.

For example:

User: What is hemoglobin?

↓

HealthAI: Explains hemoglobin.

↓

User: What happens if it is low?

The system could understand that "it" refers to hemoglobin from the previous conversation.

8.10 Secure Cloud Data Management

Future versions can introduce stronger cloud-based storage and access-control mechanisms for user reports.

Potential improvements include:

- Encrypted data storage
- Secure file access
- User-specific permissions
- Automatic session management
- Secure deletion

- Data-retention controls
- Audit logging

These measures would become increasingly important if the system were deployed for real-world healthcare use.

8.11 Advanced PDF Reports

The PDF-generation module can be enhanced to produce more professional documents.
Future PDF reports could include:

- HealthAI branding
- Report date
- Report type
- Selected language
- Structured result sections
- Tables where appropriate
- Original report reference
- Page numbers
- Document metadata

The PDF could also provide a clear separation between extracted information and AI-generated explanation.

8.12 Automatic Report Classification

Future versions can automatically identify the type of uploaded medical report.
Possible categories include:

- CBC
- Liver function test
- Kidney function test
- Lipid profile
- Blood glucose report
- Urine test
- Thyroid test
- X-ray
- Other diagnostic reports

This would reduce the need for the user to manually select the report category.

8.13 Improved Uncertainty Detection

One of the important future improvements is better identification of uncertain information.
The system could assign an internal confidence level to extracted values.
For example:
High Confidence – Clearly extracted value
Medium Confidence – Possible OCR uncertainty
Low Confidence – Value requires manual verification

This would help prevent unclear OCR results from being treated as reliable medical information.

8.14 Notification and Reminder Features

Future versions could provide optional reminders related to saved reports or user-defined health activities.

For example, the system could remind users to review a previously saved report or follow a schedule provided by their healthcare professional.

Such features should remain user-controlled and should not present AI-generated reminders as medical prescriptions.

8.15 Accessibility Improvements

Accessibility can be improved through:

- Larger text options
- Better keyboard navigation
- Screen-reader compatibility
- Voice interaction
- Improved colour contrast
- Simplified layouts
- Accessible error messages

These improvements can make HealthAI easier to use for a wider range of users.

8.16 Mobile Application

The current system is implemented as a web application.

A future version could provide dedicated Android and iOS applications.

A mobile version could provide:

- Camera-based report capture
- Direct image upload
- Mobile notifications
- Voice interaction
- Faster access to report history
- Mobile-friendly PDF sharing

8.17 Healthcare Professional Integration

A future version could provide an optional mechanism for users to share reports with healthcare professionals.

The workflow could be:

User Report

↓

HealthAI Analysis

↓

User Review

↓

Secure Sharing

↓

Healthcare Professional

The AI-generated information should remain clearly separated from professional medical interpretation.

8.18 Scalability Improvements

If the number of users increases, the backend architecture can be expanded to support larger workloads.

Possible improvements include:

- Cloud deployment
- Load balancing
- Database optimization
- Caching
- Queue-based processing
- Monitoring
- Automatic scaling

These changes would improve the ability of the system to support a larger number of simultaneous users.

8.19 Security and Privacy Enhancements

Because medical reports may contain sensitive personal information, future development should place greater emphasis on privacy and security.

Potential improvements include:

- End-to-end encryption where appropriate
- Stronger authentication
- Role-based access control
- Secure file storage
- Automatic session expiration
- Privacy controls
- Data deletion options
- Security auditing

Before real-world healthcare deployment, the system would also need to be evaluated against applicable legal, regulatory, and institutional requirements.

8.20 Clinical Validation

The current HealthAI system is an academic prototype intended for general health information and report understanding.

Before clinical use, the system would require extensive validation.

This could include:

- Testing with diverse medical reports
- Evaluation of OCR accuracy
- Evaluation of AI response accuracy

- Medical expert review
- Image-analysis validation
- Safety testing
- Bias evaluation
- Privacy and security assessment

Clinical validation would be necessary before relying on the system for healthcare decision-making.

8.21 Future Development Roadmap

The proposed future development can be organized into stages.

Phase 1 – Accuracy

- Improve OCR
- Improve document classification
- Improve uncertainty detection

Phase 2 – User Experience

- Voice interaction
- Accessibility improvements
- Mobile application
- Additional languages

Phase 3 – Advanced Analysis

- Longitudinal report comparison
- Improved image analysis
- Structured medical data extraction

Phase 4 – Security and Deployment

- Advanced cloud security
- Scalable infrastructure
- Privacy controls
- Professional validation

8.22 Chapter Summary

This chapter presented the potential future enhancements of the HealthAI system.

The major opportunities include improved OCR accuracy, advanced medical-image processing, longitudinal report comparison, additional languages, voice interaction, personalized information, stronger security, improved PDF generation, automatic report classification, accessibility improvements, mobile applications, and healthcare-professional integration.

These enhancements can improve the usability, scalability, reliability, and accessibility of HealthAI while maintaining the principle that AI-generated information should not replace qualified professional medical evaluation.

CHAPTER 9

CONCLUSION

9.1 Introduction

The HealthAI project was developed as an AI-assisted web application for simplifying the understanding of medical reports and providing general health information.

The system combines report processing, OCR, AI-based analysis, medical image observation, chatbot interaction, multilingual support, report history, and PDF generation within a single application.

9.2 Project Outcome

The developed system successfully demonstrates the proposed workflow for processing medical information.

Users can authenticate into the application, upload supported medical reports, extract available information, and obtain an AI-generated explanation.

The system also provides a conversational interface through which users can ask general health-related questions.

The implementation demonstrates how different software components can be integrated to create a unified healthcare-information platform.

9.3 Major Achievements

The major achievements of the HealthAI project are:

1. Developed a functional web-based healthcare information application.
2. Implemented user authentication.
3. Implemented medical report upload functionality.
4. Integrated OCR-based text extraction for image-based reports.
5. Implemented image preprocessing before OCR.
6. Integrated AI-based medical report explanation.
7. Added medical image and X-ray observation functionality.
8. Developed the HealthAI chatbot.
9. Added English and Tamil language support.
10. Implemented report history.
11. Implemented downloadable PDF analysis.
12. Added input validation and error handling.
13. Integrated frontend and backend processing into a unified workflow.

9.4 Significance of the Developed System

The primary contribution of HealthAI is the integration of several functions that would otherwise require separate tools.

A user can upload a report, obtain extracted information, receive an AI-assisted explanation, interact with the chatbot, review previous analyses, and generate a downloadable document through the same application.
The system therefore demonstrates the practical application of AI, OCR, web development, and image processing within a healthcare-information context.

9.5 Limitations

Although the project demonstrates the intended functionality, certain limitations remain.
OCR accuracy depends on the quality of the uploaded document.
AI-generated explanations depend on the information available in the input and may not always represent complete clinical context.
Medical image observations are limited by image quality and should not be considered confirmed diagnosis.
The current system is an academic prototype and has not been validated for independent clinical decision-making.

9.6 Future Scope

The system can be further improved through:

- Advanced OCR
- Improved medical image analysis
- Automatic report classification
- Longitudinal report comparison
- Additional Indian languages
- Voice-based interaction
- Mobile applications
- Improved accessibility
- Stronger security and privacy controls
- Healthcare professional integration
- Clinical validation

These enhancements can improve the system's reliability, usability, scalability, and accessibility.

9.7 Final Conclusion

HealthAI demonstrates the feasibility of developing an integrated AI-assisted platform for medical information understanding.
The project successfully combines **web application development, OCR, artificial intelligence, medical image processing, authentication, multilingual interaction, history management, and PDF generation** into a single system.
The developed application is intended to make medical information easier to understand while maintaining an important distinction between **AI-generated general information and professional medical diagnosis**.

Overall, the project provides a practical foundation for future development of intelligent and user-friendly healthcare information systems.

CHAPTER 10

REFERENCES

10.1 References

The following resources were referred to during the design, development, implementation, and documentation of the HealthAI application.

Web Technologies and Frameworks

[1] React Documentation, "React – The library for web and native user interfaces," Meta Platforms, Inc.

[2] Node.js Documentation, "Node.js – JavaScript runtime environment," OpenJS Foundation.

[3] Express.js Documentation, "Express – Node.js web application framework," OpenJS Foundation.

[4] Tailwind CSS Documentation, "Tailwind CSS – Utility-first CSS framework."

[5] React Router Documentation, "React Router – Declarative routing for React applications."

Authentication and Database

[6] Firebase Documentation, "Firebase Authentication," Google.

[7] Firebase Documentation, "Cloud Firestore," Google.

[8] Firebase Documentation, "Firebase Web SDK Documentation," Google.

OCR and Document Processing

[9] Tesseract.js Documentation, "Tesseract.js – JavaScript OCR library."

[10] Tesseract OCR Documentation, "Tesseract OCR Engine."

[11] Mozilla PDF.js Documentation, "PDF.js – PDF rendering and processing."

Artificial Intelligence

[12] Groq Documentation, "Groq API Documentation."

[13] Groq Documentation, "Groq SDK and API Reference."

[14] OpenAI, "OpenAI API Documentation," for general reference on large language model interaction and API-based AI systems.

Frontend Libraries

[15] React Markdown Documentation, "React Markdown – Markdown component for React."

[16] MDN Web Docs, "Fetch API – Web APIs."

[17] MDN Web Docs, "Web Storage API."

Programming and Development References

[18] MDN Web Docs, "JavaScript Guide."

[19] MDN Web Docs, “HTML and CSS Web Development Documentation.”
[20] GitHub Documentation, “GitHub – Development and Version Control Platform.”

10.2 Project Development References

The implementation of HealthAI also relied on the official documentation and technical references associated with the software components used in the project.
These references were used primarily to understand:

- API integration
- Authentication implementation
- React application development
- Backend API development
- OCR processing
- PDF processing
- AI service integration
- Frontend response rendering
- Client-server communication

10.3 Reference Usage

The references were used during different stages of the project.

Reference Area	Application in HealthAI
React Documentation	Frontend development
Node.js Documentation	Backend runtime
Express Documentation	REST API implementation
Firebase Documentation	Authentication
Tesseract.js Documentation	OCR implementation
PDF.js Documentation	PDF processing
Groq Documentation	AI integration
React Markdown Documentation	AI response formatting
MDN Documentation	JavaScript and Web APIs
GitHub Documentation	Development and version control

10.4 Chapter Summary

This chapter presented the technical and documentation references used during the development of the HealthAI application.
The references primarily consist of official documentation for the frameworks, libraries, APIs, and development technologies used in the project.